arXiv:2602.03128v2 [cs.AI] 17 Sep 2026

Architectural Design, Not Only Model Intelligence, Governs Multi-Agent LLM Performance

Abdelghny Orogat
Concordia University

Ana Rostam
Concordia University

Essam Mansour
Concordia University

Abstract

Multi-agent LLM frameworks are data-intensive systems that govern how agents orchestrate tasks, manage state, and coordinate decisions. These architectural choices control execution overhead, memory behavior, planning effectiveness, and coordination scalability. Their impact on system performance remains poorly understood. Existing benchmarks evaluate individual agent capabilities in isolation and lack standardized framework-level comparison. We make four contributions. We introduce an architectural taxonomy that decomposes multi-agent LLM frameworks along five dimensions: orchestration, memory, planning interfaces, specialization, and communication topology. We develop MAFBench, a unified evaluation suite that integrates existing benchmarks within a standardized execution pipeline. We conduct a controlled empirical study across nine frameworks, fixing the underlying LLM and varying only architectural design choices. We distill the results into six evidence-based design principles. Architectural design, not only model intelligence, governs performance. Orchestration alone increases latency by over 60×, and a minimal implementation of the same paradigm isolates that cost as implementation rather than paradigm. Schema-constrained planning interfaces reduce accuracy by up to 32 points through formatting failures, not reasoning errors. Communication topology drops coordination success from above 90% to below 30% under mismatched structure. Memory architecture controls recall and scalability independent of context window size, and no evaluated framework natively supports controlled knowledge revision.

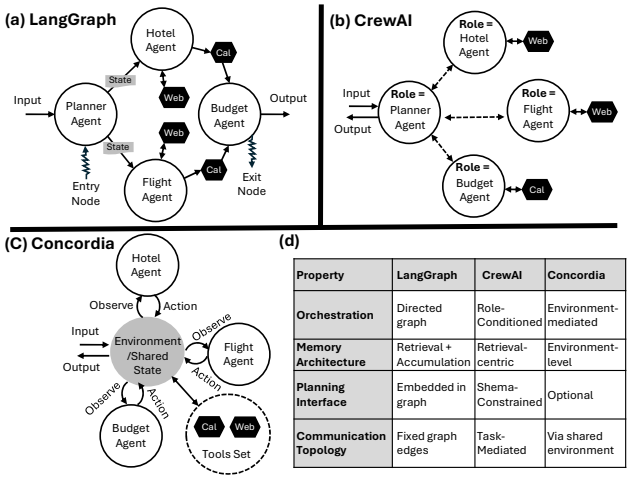
1 Introduction

Multi-agent LLM frameworks govern how agents orchestrate tasks, manage state, and coordinate decisions in systems powered by large language models (LLMs) [8, 29, 50, 57, 66]. A growing ecosystem of such frameworks has emerged to support complex intelligent applications [19, 26, 31, 44, 47, 49, 59, 62]. These frameworks function as data-intensive systems. Their architectural choices determine orchestration structure, storage design, planning interfaces, and communication topology. These choices shape system behavior much like query processing pipelines, storage engines, and distributed coordination protocols shape database performance.

This paper evaluates the framework orchestration layer, not the applications built on top of it. Their impact on system performance remains poorly understood. We evaluate five architectural dimensions: orchestration structure, memory architecture, planning interfaces, specialization conditioning, and communication topology. Table 1 reports the performance ranges across these dimensions at fixed LLM and task. Architectural design alone induces order-of-magnitude differences in latency and throughput, and substantial variation in accuracy and task success. System behavior is therefore governed primarily by architectural design, not LLM quality alone.

Table 1: Performance ranges across architectural design choices at fixed task and LLM. Residual is framework time outside the model call, and it separates paradigm cost from implementation cost.

Dimension	Metric	Best	Worst
Orchestration	Latency (× direct LLM)	1.1×	62×
	Throughput (req/s)	7.0	0.12
	Residual (% latency)	< 0.1	91.5
Memory	Overall score (%)	23.8	6.1
Planning	Accuracy (pts vs. no plan)	+15.7	−32.3
	Runtime (× no plan)	0.8×	31.3×
Specialization	ΔF1 (vs. no role)	+58	≈ 0
Topology	Success, large n (%)	> 90	< 30



(d)

Property	LangGraph	CrewAI	Concordia
Orchestration	Directed graph	Role-Conditioned	Environment-mediated
Memory Architecture	Retrieval + Accumulation	Retrieval-centric	Environment-level
Planning Interface	Embedded in graph	Shema-Constrained	Optional
Communication Topology	Fixed graph edges	Task-Mediated	Via shared environment

Figure 1: Running example: the same task (trip planning) instantiated under three paradigms (a–c). Panel (d) compares their design choices along the architectural dimensions evaluated in this study. *Web (Search)/Cal(Calculator)* denote tools.

To illustrate these differences, we consider a simple task: planning a travel itinerary with flights, accommodation, and a budget constraint. Figure 1 instantiates it under the three general-purpose paradigms of Table 2. Task-specific frameworks fix orchestration to one task family, so we treat them separately (Section 2.1). LangGraph [31] encodes orchestration as an explicit workflow graph. CrewAI [26] organizes agents through role-based delegation. Concordia [59] mediates interaction through a shared environment. Concordia is the only production GABM framework, so we add a second data point: a *minimal GABM skeleton* with the same environment-mediated structure but no simulation machinery. This makes two gaps measurable. Skeleton versus Concordia isolates

Table 2: Representative frameworks by paradigm, with the entity holding control-flow authority in *italics*. • marks frameworks evaluated here. The skeleton gives the GABM paradigm a second data point.

Paradigm	Framework	Stars	Forks	Contrib.	Eval.
Graph *(developer)*	LangGraph [31]	18.9k	3.3k	249	•
	n8n [42]	140k	44k	400+	
	Langflow [32]	19.6k	2.1k	120	
Role *(agents)*	CrewAI [26]	38.4k	5.1k	274	•
	AutoGen [62]	50.0k	7.7k	558	•
	OpenAI SDK [47]	14.9k	2.5k	166	•
	Agno [3]	4.2k	310	40	•
	OpenAgents [63]	1.1k	143	5	•
GABM *(environment)*	Concordia [59]	1.0k	218	<50	•
	GABM skeleton (ours)	—	—	—	•
Task-specific *(fixed)*	Claude Code [5]	142k	22k	57	
	Cursor [6]	33k	2.3k	34	
	Cline [27]	65.4k	7.0k	334	
	OpenHands [4]	82.2k	10.7k	538	•
	OpenClaw [13]	384k	80.7k	1.6k	

implementation cost. Skeleton versus the other paradigms isolates the cost of environment mediation. The task, agents, and LLM stay fixed, so each paradigm differs only in execution structure.

Prior work examines individual aspects of multi-agent intelligence: planning [12, 21, 56], memory and retrieval [23, 25, 36, 65, 68], tool use [18, 41, 43, 54], and collaborative reasoning [17]. These studies target model-level capabilities, not framework-level architectural choices. Three limitations persist: (i) no characterization of architectural dimensions and their role in system behavior, (ii) no unified suite for cross-framework comparison, and (iii) no controlled evidence isolating individual design choices.

We address these limitations with four contributions. Our study design is controlled. We fix the underlying LLM and vary only architectural design choices, so observed differences follow from architecture and not from model capability. Planning is evaluated across ten LLMs because the interface effect interacts with capability. All other dimensions hold the model constant. One paradigm ships with a single production framework, so we implement a minimal reference version of it to separate paradigm cost from implementation cost. Our main contributions are:

- An architectural taxonomy that compares multi-agent LLM frameworks along five dimensions: orchestration, memory architecture, planning interfaces, specialization conditioning, and communication topology. The taxonomy raises five research questions that guide our evaluation (Section 2).
- **MAFBench**[1], a standardized evaluation suite that integrates existing benchmarks within a unified execution and scoring pipeline for controlled framework-level comparison (Section 3).
- A controlled empirical study across nine frameworks and five architectural dimensions. It shows that architectural design, not model intelligence alone, governs performance, scalability, and cost (Section 4).
- Six evidence-based design principles that guide the construction and selection of multi-agent LLM frameworks. Each principle states where it applies and where it does not (Section 5).

2 Architectural Taxonomy

This section introduces our architectural taxonomy for multi-agent LLM frameworks. Architectural design choices shape control flow, agent abstraction, communication, memory, and execution semantics. They have first-order impact on performance, robustness, and scalability. The taxonomy enables framework-level comparison independent of specific models or tasks. We instantiate the trip-planning example of Figure 1 in each paradigm and report its execution trace alongside the corresponding definition. We begin with formal definitions.

Definition 2.1 (Agent). An agent is an autonomous computational entity $a = (\mathcal{R}, \mathcal{Y}, \mathcal{P}, \mathcal{S}, \mathcal{T}, f)$, where $\mathcal{R}$ is role context, $\mathcal{Y}$ is objectives, $\mathcal{P}$ is planning mechanisms, $\mathcal{S}$ is storage and knowledge resources, $\mathcal{T}$ is tools or actions, and f is a reasoning function mapping observations and stored state to actions. We focus on LLM-based agents, so f is an LLM. Model-native agentic capabilities, such as tool calling and internal planning, are part of f. Our taxonomy targets the framework-level abstractions that wrap and constrain f.

Definition 2.2 (General-Purpose Multi-Agent LLM Framework). A general-purpose multi-agent LLM framework is an architectural system $\mathcal{F} = (\{a_i\}_{i=1}^{n}, \mathcal{O}, \mathcal{C}, \mathcal{E})$, where $\{a_i\}$ is a set of LLM-based agents, $\mathcal{O}$ is orchestration logic and control flow, $\mathcal{C}$ is communication structure and connectivity, and $\mathcal{E}$ is the shared environment whose state and transition rules mediate agent interactions.

Definition 2.3 (Task-Specific Multi-Agent Framework). A task-specific multi-agent framework instantiates a system-supplied orchestration loop $\mathcal{O}_{\text{fixed}}$ for one task family. It spawns sub-agents through a delegation primitive $d : a \to \{a\}$ that induces a rooted tree over $\{a_i\}$. Extensibility augments $\mathcal{T}$, $\mathcal{R}$, and $\mathcal{P}$ through tools, sub-agent definitions, planning modes, and hooks. Communication $\mathcal{C}$ is limited to delegation and return, and no shared environment $\mathcal{E}$ is exposed.

Coding assistants such as Claude Code [5], Cursor [6], Cline [27], and OpenHands [4] instantiate Definition 2.3 (Table 2). Each supports sub-agent delegation but fixes $\mathcal{O}$ to a software-development loop. OpenClaw [13] fixes $\mathcal{O}$ to a personal-assistant loop over messaging channels, so it instantiates the same definition over a different task family. This paper studies general-purpose frameworks (Definition 2.2). They expose orchestration, communication, and storage as design choices, so they admit controlled variation of one dimension at a time.

Table 3 presents the resulting taxonomy on the representative systems of Table 2. It organizes design choices along two axes: architectural paradigms (Section 2.1) and architectural dimensions (Section 2.2). Section 2.3 then states the research questions that guide our evaluation.

2.1 Architectural Paradigms

Each paradigm is a coherent class of design commitments that jointly fix $\mathcal{O}$, $\mathcal{C}$, agent abstraction, and $\mathcal{E}$. Paradigms separate along a single axis: which entity holds control-flow authority, the developer at design time, the agents at runtime, or the environment through mediation. Exactly one entity holds it, so the categories are mutually exclusive by construction. Table 2 maps the evaluated frameworks onto this axis. Task-specific frameworks fall outside it. The vendor composes authored control flow with role-conditioned delegation into a loop for one task family and ships it fixed. The developer inherits this composition, and defines sub-agents only within the slots the shipped loop provides (Definition 2.3). The running example fixes one request (Boston → New York, four days, $500) and four roles, *Planner*, *Flight*, *Hotel*, and *Budget*, with a search

[1] https://github.com/CoDS-GCS/MAFBench

Table 3: Architectural taxonomy of multi-agent LLM frameworks across graph-based, role-based, and GABM paradigms. It decomposes frameworks into core design dimensions to enable systematic, framework-level comparison independent of models.

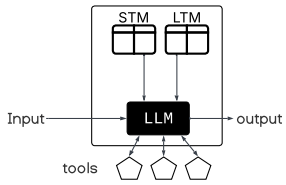
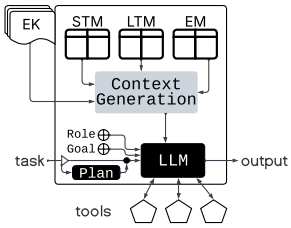
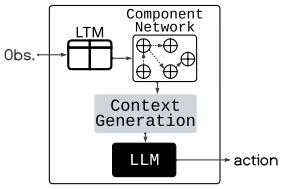
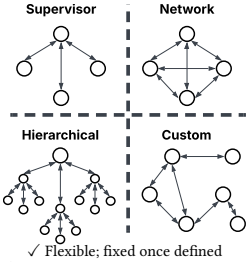
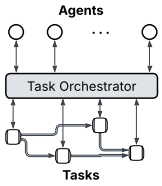
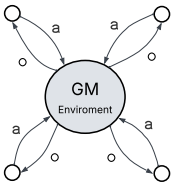
Feature	Graph-Based (LangGraph)	Role-Based (CrewAI)	GABM (Concordia)
1. Single-Agent Characteristics			
Main Purpose	Deterministic workflow orchestration via explicit execution graphs; agent-based	Task-driven coordination via role-specialized agents and delegation; task-based	Simulation of emergent behavior through environment-mediated agent interaction
Agent Architecture			
Behavioral Specification			
Role	✗ Implicit; No first-class role abstraction	✓ Explicit; First-class role abstraction	✓ Optional; Provided class targets social experiments
Goal	✗ Determined by graph; No first-class goal object	✓ First-class object; fixed; one per agent	✗ Environment-driven; No first-class goal object
Planning	✗ Embedded in graph; LLM planning requires coding	✓ Pre-task or manager-worker planning	✓ Optional; Provided class targets social experiments
Storage			
Long-Term Memory (LTM)	✓ Retrieval-based semantic store	✓ Retrieval-based semantic store	✓ LLM-queried textual memory
Short-Term Memory (STM)	✓Conversation or state accumulation within execution	✓ RAG-based working context over recent interactions	✗ Not natively supported
Entity Memory (EM)	✗ Not natively supported	✓ Optional; structured entity facts via retrieval	✗ Not natively supported
Working Memory (WM)	✗ State variables must be managed by developer	✗ Implicit via task context; not explicitly modeled	✓ Derived from LTM; update rules by developer
External Knowledge (EK)	✗ Not natively supported	✓ Explicitly attachable (files, strings)	✗ Not natively supported
Tool Execution	✓ Agent-bound tools or explicit graph tool nodes	✓ Agent-bound tools	✓ Tools executed by environment, not by agents
2. Multi-Agent Characteristics			
Network Topology			
Communication Pattern	✓ Flexible; fixed once defined	✓ Task hierarchy or sequence	✓ Fixed star topology; GM at center
Collaboration	✓ Shared state along predefined edges ✗ None; procedural coordination only	✓ Task-mediated; limited peer querying ✓ Partial; delegation-based coordination only	✗ Only via GM; no peer-to-peer ✗ None; no explicit collaboration mechanisms
3. Environment			
World State and Grounded Variables	✗ None; execution context only	✗ None; task-centric context only	✓ Explicit world state with grounded variables

Table 4: Paradigm traces on the running example (*gpt-4o-mini*). Only orchestration varies. *Prompts* is whether every issued prompt is retrievable. CrewAI spends a second model turn on each tool result. The skeleton matches LangGraph on calls but carries environment state in every observation.

Paradigm	Control flow	Prompts	LLM calls	Tokens (in/out)
LangGraph (graph node)	graph edges	full	4	138 / 45
CrewAI (agent-bound)	role hand-off	partial	7	2,677 / 463
GABM skeleton (env)	env-mediated	full	4	296 / 204

tool (*Web*) and a calculator (*Cal*). We implement it in *LangGraph*, *CrewAI*, and the *GABM skeleton*, holding task, tools, and model (*gpt-4o-mini*) fixed. One instrumented client calls the model, so calls and tokens are comparable (Table 4).[2] Invocation is deterministic wherever the paradigm permits, so the traces isolate orchestration. Section 2.2.1 varies it instead.

2.1.1 Graph-Based Frameworks. Graph-based frameworks encode orchestration O as an explicit directed graph. Nodes represent agents or computational modules, and edges define permissible control or data flows [31, 32, 42]. Execution follows this predefined structure (Figure 1.a). The developer holds control-flow authority.

[2]Full traces are provided in the supplementary materials.

In our LangGraph implementation, trip state advances along author-defined edges and each agent runs once, completing the task in four LLM calls (Table 4). The paradigm admits two invocation modes, and Figure 1.a shows both: *Web* bound to an agent and *Cal* as a graph node. The trace places both on graph nodes, so no model call carries a tool schema and every prompt is visible. Section 2.2.1 varies tool invocation.

Definition 2.4 (Graph-Based Framework). A graph-based framework instantiates O as a directed graph $\mathcal{G} = (V, E)$, where $V = \{a_1, \ldots, a_n\}$ are agents or components and $E \subseteq V \times V$ defines allowable control or data flows. Communication C propagates only along edges in $\mathcal{G}$. No explicit environment state $\mathcal{E}$ is maintained.

2.1.2 Role-Based Frameworks. Role-based frameworks organize agents around textual role specifications that condition objectives, tool access, and interaction behavior [3, 26, 47, 62, 63]. Coordination emerges through role-conditioned reasoning, delegation, and structured message exchange (Figure 1.b). The agents hold control-flow authority. In our CrewAI implementation, the Planner delegates to the Flight, Hotel, and Budget roles at runtime. CrewAI binds tools to agents and offers no graph-node alternative, so each specialist spends a second model turn consuming its tool result. The task takes seven calls with an order of magnitude more input tokens

(Table 4). Roles replace authored control flow, trading determinism for tokens. Internal delegation prompts are not retrievable, so we record task descriptions and aggregate token counts rather than every prompt.

Definition 2.5 (Role-Based Framework). A role-based framework instantiates agents $\{a_i\}_{i=1}^{n}$ with a role space D and a mapping $R : \{a_i\} \to D$ assigning each agent a role. Role-conditioned interactions govern O and C. No explicit environment state $\mathcal{E}$ is maintained.

2.1.3 GABM Frameworks. GABM frameworks treat multi-agent systems as simulations where behavior emerges from agent–environment interactions rather than explicit orchestration. Coordination is mediated through shared world state, updated by a transition function T (Figure 1.c). The environment holds control-flow authority, and the realization of T is a design choice within the paradigm. Concordia [59] realizes T through the LLM. A game master interprets each agent action in narrative form and resolves its effect, adding model calls beyond those of the agents. Our skeleton realizes T deterministically in code, applying a validated action without consulting the model. This bounds a round at one call per agent and completes the example in four calls with every prompt visible (Table 4). At equal call count it costs more input tokens than LangGraph, because each observation carries environment state where a graph edge passes only what the next node needs. Cost follows the realization, not the paradigm.

Definition 2.6 (GABM Framework). A GABM framework instantiates agents $\{a_i\}_{i=1}^{n}$ interacting with an environment state $\mathcal{E}$ governed by a transition function $T : \mathcal{E} \times \{a_i\} \to \mathcal{E}$. Orchestration O arises from environment-mediated interaction. Direct inter-agent communication C is not exposed.

2.2 Architectural Design Decisions

We organize architectural design decisions into three groups: single-agent characteristics (behavioral specification, storage architecture, tool execution), multi-agent characteristics (network topology, communication pattern, collaboration), and environment representation. Each group corresponds to a set of rows in Table 3, and each row captures a distinct axis along which frameworks differ. The columns instantiate these dimensions for the three paradigms using LangGraph (graph-based), CrewAI (role-based), and Concordia (GABM) as representative systems. The agent architecture diagrams in Table 3 show how each paradigm composes storage $\mathcal{S}$, planning $\mathcal{P}$, tools $\mathcal{T}$, and the reasoning function f into distinct execution pipelines. We describe each dimension below, then how they couple.

2.2.1 Single-Agent Characteristics. **Behavioral Specification.** This dimension describes how agent behavior is defined at runtime using roles $\mathcal{R}$, goals $\mathcal{Y}$, and planning $\mathcal{P}$ (Definition 2.1). Planning has two layers. The planning logic produces the intermediate steps and resides in f. The planning interface is the framework mechanism that decides whether a plan is elicited, what format it must satisfy, and where it enters the context. Frameworks wrap planning logic. They do not implement it [3, 26, 62]. The Behavioral Specification rows of Table 3 show how the three paradigms realize these components. Graph-based frameworks implement $(\mathcal{R}, \mathcal{Y}, \mathcal{P})$ procedurally within O, so control flow determines behavior rather than explicit abstractions. LangGraph exposes no planning interface. The authored graph is the plan, and eliciting one from the model requires developer coding (Figure 1.a) [31, 32, 42, 55]. Role-based frameworks expose $\mathcal{R}$ and $\mathcal{Y}$ as first-class textual inputs that guide f. CrewAI adds a pre-task or manager-worker interface that requests a plan from the model and appends it to the task context (Figure 1.b) [26, 47, 62, 63]. GABM frameworks treat $(\mathcal{R}, \mathcal{Y}, \mathcal{P})$ as optional. Concordia provides role and planning classes for social simulation experiments, but the environment drives goals rather than declaring them (Figure 1.c) [59]. These differences determine which frameworks expose a planning interface that can be varied under control.

Storage Architecture. This dimension captures how agent knowledge $\mathcal{S}$ is represented, accessed, and updated during execution. Let $\mathcal{S} = \{\text{LTM}, \text{STM}, \text{EM}, \text{WM}, \text{EK}\}$ denote the memory types exposed by frameworks. Each $s \in \mathcal{S}$ has four properties: lifetime (short-lived vs. persistent), structure (unstructured vs. structured), access mode (retrieval, context accumulation, or state update), and ownership (agent-local vs. environment-level). The Storage rows of Table 3 show which types each paradigm realizes natively. LangGraph supports retrieval-based LTM and accumulation-based STM. CrewAI adds EM and EK. Concordia centralizes storage in $\mathcal{E}$ and derives WM from LTM. Every paradigm leaves at least two types unsupported, so the available access mechanisms are retrieval, accumulation, and their hybrid. Memory types requiring custom implementation are excluded.

Tool Execution Model. This dimension determines how external actions in $\mathcal{T}$ are invoked. The Tool Execution row of Table 3 shows three realizations: agent-bound tools or graph tool nodes constrained by O in graph-based frameworks (*Web* and *Cal* in Figure 1.a), agent-bound tools through role assignment in role-based ones (Figure 1.b), and execution routed through $\mathcal{E}$ in GABM frameworks, where agent outputs become state updates (Figure 1.c).

Every paradigm delegates tool selection to f, so the architectural property is not who selects but whether invocation is guaranteed. A graph tool node and a mediator applying an action invoke deterministically. Authoring the node and its position in O costs developer effort, but guarantees execution and saves the tokens spent transmitting schemas and deciding invocation. Agent-bound invocation carries no such guarantee. Two instructions for the same LangGraph configuration give opposite outcomes under *gpt-4o-mini*. A permissive one offering the tool as optional never calls it and yields no valid plan. A directive one naming it as the way to obtain prices invokes it and completes. CrewAI uses the directive form and re-enters the model after each tool result, biasing its loop toward invoking, and *Opus 5* invokes under both.[3] Invocation depends on wording, agent loop, and model at once, so measuring agent-bound selection would rank the model rather than isolate a design dimension.

2.2.2 Multi-Agent Characteristics. The bottom rows of Table 3 define multi-agent interaction through three aspects: network topology (the structure of $C \subseteq \{a_i\} \times \{a_i\}$), communication pattern (how information is exchanged over C), and collaboration (how agents

[3] Per-configuration invocation results are in the supplementary materials.

work toward shared goals). The paradigms differ on all three. Graph-based frameworks define C explicitly at design time and fix it during execution, so communication is shared state along predefined edges (Figure 1.a) [31, 42, 55]. Role-based frameworks induce C from roles $\mathcal{R}$ and objectives $\mathcal{Y}$, so communication is task-mediated and collaboration is delegation-based (Figure 1.b) [26, 47, 62]. GABM frameworks remove direct agent connections ($C = \emptyset$) and route everything through a Game Master at the center of a fixed star topology (Figure 1.c) [59]. Neither graph-based nor GABM frameworks provide explicit collaboration mechanisms. These structural constraints determine how information flows between agents and whether coordination scales with the number of agents.

2.2.3 Environment. This dimension describes how execution is mediated through shared world state $\mathcal{E}$ (Definition 2.2). The Environment row of Table 3 shows that only GABM frameworks expose one. Graph-based frameworks operate on execution context, where intermediate outputs propagate along workflow edges. Role-based frameworks maintain task-centric context without persistent global state. GABM frameworks define an explicit $\mathcal{E}$ with grounded variables that persist across rounds, updated by a transition function $T : \mathcal{E} \times \{a_i\} \to \mathcal{E}$ (Definition 2.6) [59]. The presence or absence of $\mathcal{E}$ determines whether coordination state survives across rounds. Frameworks without $\mathcal{E}$ must re-derive shared context from message history, but GABM frameworks accumulate it in persistent world state.

2.2.4 Dimension Coupling and Evaluation Scope. Choosing a paradigm fixes some dimensions and leaves others open, so they are not independent. A dimension is *fixed* when the paradigm or framework determines it, and *variable* when the developer selects it, whether freely or from natively supported options. Orchestration O and communication C are coupled throughout. The execution graph is also the communication structure, the role mapping $\mathcal{R}$ induces both and binds $\mathcal{T}$ to agents, and $\mathcal{E}$ mediates C and owns $\mathcal{S}$. Controlled variation needs one variable dimension while the rest stay fixed. We therefore vary topology in graph-based frameworks, storage in graph-based and role-based ones, planning interface in CrewAI, and tools in LangGraph. Varying a definitional dimension would change the paradigm rather than isolate it, so some framework–dimension combinations are absent from the evaluation grid. Task-specific frameworks seal O, C, and $\mathcal{S}$ inside the system loop, leaving only $\mathcal{P}$, $\mathcal{R}$, and $\mathcal{T}$. Their planning offers the same interfaces we already evaluate. A full factorial study and a dedicated study of task-specific systems follow directly from this scoping.

2.3 Research Questions

The taxonomy raises one open question per dimension, each derived from the design differences above and answered in Section 4.

RQ1 (Memory Access Mode). Which access mode governs multi-hop retrieval (RQ1a), long-range understanding (RQ1b), and in-session learning (RQ1c)? Does hybrid retrieval–accumulation behave like either component alone? And what does each architecture support under knowledge revision (RQ1d), a state update rather than an append?

RQ2 (Planning Interface). How does the output format a planning interface imposes on $\mathcal{P}$ affect accuracy and runtime, and does the effect come from reasoning or parsing failures?

RQ3 (Specialization). Does a role activate domain reasoning by naming an identity or by binding a capability, and does binding require procedure?

RQ4 (Communication Topology). How does connectivity in C affect coordination as agents grow in number, and what connectivity do different tasks need within bounded rounds?

RQ5 (Orchestration Overhead). What execution cost does each orchestration structure impose on a trivial task, and how much of the cost of mediation through $\mathcal{E}$ is inherent to the paradigm rather than a framework's implementation?

3 MAFBench: A Unified Benchmark

This section introduces **MAFBench**, a unified benchmark for evaluating multi-agent frameworks across the architectural dimensions in Section 2.

MAFBench Contributions. Existing benchmarks evaluate individual agent capabilities in isolation, not framework-level design choices. MAFBench compares frameworks as deployed, standardizing everything around the framework and leaving the framework itself untouched. It contributes four pieces of infrastructure. A *unified execution pipeline* wraps heterogeneous frameworks behind a common interface, enforcing identical prompts and stopping criteria. *LLM-based semantic scoring* replaces brittle string matching and handles diverse answer formats. *Configuration parity* centralizes model selection, context limits, batching, and concurrency, so observed differences reflect architecture rather than drift. Every framework exposing decoding gets the same setting within an experiment, at or near temperature 0. Frameworks exposing none use provider defaults, and no seed is set. *Cost control and backend routing* redirect compatible API calls to alternative providers without modifying framework code. Where a paradigm has one production framework, its measurements carry that implementation's idiosyncrasies, so we add a minimal reference implementation to separate paradigm cost from implementation cost (Section 4.1.1). Each subsection below adds only what its dimension requires.

3.1 Memory Benchmarks

Prior memory benchmarks for LLM-based agents evaluate isolated capabilities: long-context retrieval [23, 61, 67], dialogue and state tracking [36, 38], incremental preference learning [9, 20, 33], and controlled knowledge revision [68]. None captures how agents retain, integrate, generalize, and revise information across multi-turn interactions. We therefore adopt *MemoryAgentBench* [25], which unifies these dimensions into four core competencies.[4]

Benchmark Unified Interface. MemoryAgentBench organizes evaluation into four competency-aligned splits composed of adapted subtasks (Table 5). *Accurate Retrieval (AR)* measures factual recall and multi-hop reasoning over long contexts. *Test-Time Learning (TTL)* evaluates in-session learning of concepts and preferences. *Long-Range Understanding (LRU)* targets long-range abstraction and cross-document reasoning. *Selective Forgetting (SF)* probes controlled memory revision through counterfactual updates [25].

Evaluation Setup. We standardize document ingestion using fixed-size text chunking with controlled overlap, consistent ordering,

[4] https://huggingface.co/datasets/ai-hyz/MemoryAgentBench

Table 5: MemoryAgentBench statistics per competency split. Each split evaluates a distinct memory capability: Accurate Retrieval (AR), Test-Time Learning (TTL), Long-Range Understanding (LRU), and Selective Forgetting (SF). Question counts and context lengths are in thousands of words.

			Questions				Context (k words)			
Split	Sessions	Subtasks	Min	Max	Avg	Total	Min	Max	Avg	Total
AR	22	4	60	100	90.9	2,000	50	560	206	4,551
TTL	6	2	100	200	116.7	700	69	1,040	236	1,417
LRU	110	2	1	10	1.6	171	48	560	129	14,281
SF	8	2	100	100	100.0	800	4	183	63	511

Table 6: Planning benchmark statistics. GSM8K covers numerical reasoning, CSQA covers commonsense reasoning, and MATH covers symbolic reasoning.

		Question Length (words)				Gold Answer Length (words)			
Benchmark	#Q	Min	Max	Avg	Total	Min	Max	Avg	Total
GSM8K [12]	1319	15	164	46.3	61,005	1	1	1.0	1,319
CSQA [56]	1221	19	69	31.6	38,615	1	1	1.0	1,221
MATH [21]	5000	2	252	30.7	153,261	1	19	1.3	6,345

and identical session initialization across frameworks. The suite is long-context, so we route memory-intensive workloads to Groq-hosted models [2] through transparent backend routing. This lowers evaluation cost without modifying framework implementations.

Scope: framework-native memory. We evaluate the memory architectures the frameworks provide themselves. Plugin layers such as Mem0 [10], Letta [45], Zep [53], MIRIX [60], and EverMemOS [24] attach to any framework, so they do not differentiate frameworks along the dimension we study. MAFBench's adapter interface holds the framework fixed while varying the memory backend, so it enables their in-depth evaluation.

3.2 Planning Interface Benchmarks

This study varies the planning interface (Section 2.2.1) and holds the planning logic fixed. The same model generates the plan under every mode. CrewAI [26] is the only evaluated framework exposing a runtime planning interface, so we compare it against no interface and against a free-form interface injected outside the framework. Evaluation of these interfaces remains fragmented across reasoning benchmarks. We adopt *GSM8K* [12], *CommonsenseQA (CSQA)* [56], and *MATH* [21] (Table 6), which cover numerical, commonsense, and symbolic reasoning tasks where multi-step planning influences outcomes.[5]

Benchmark Planning Interfaces. Each benchmark runs under three controlled interfaces. *NoPlan* passes tasks directly to the LLM without eliciting a plan. *Crew-Plan* follows CrewAI's schema-constrained two-stage interface [26], where the framework requests a structured plan from the LLM and appends it to the task description. *Direct-LLM-Plan* elicits an unconstrained natural-language plan from the same model and injects it into the context before answer generation. The three modes differ in whether a plan is requested, what format it must satisfy, and where it enters the context. They do not differ in which model produces the plan.

[5]https://huggingface.co/datasets/openai/gsm8k (test only); https://huggingface.co/datasets/tau/commonsense_qa (validation only); https://github.com/hendrycks/math

Table 7: Specialization benchmark datasets from CatDB [16], spanning regression, binary, and multiclass classification tasks with varying feature dimensionality.

Dataset	Task Type	Target Variable	# Features
Utility	Regression	CSRI	12
WiFi	Binary Classification	TechCenter	9
EU-IT	Multiclass Classification	Position	23
Yelp	Multiclass Classification	stars	184
Volkert	Multiclass Classification	class	181

Evaluation Setup. The pipeline treats the planning interface as a configurable execution stage and supports complexity-preserving subsampling. We use a 100-problem MATH subset. Formatting failures are tracked separately from reasoning errors, and all three interfaces run across multiple LLM backends to verify that observed effects are architectural rather than model-specific. No interface verifies, critiques, or replans. Each plan is produced once and injected once, so differences come from format and injection point rather than control policy. CrewAI builds its plan internally and exposes no callback, so we report its schema-violation and runtime effects. Free-form plans are logged in full.[6]

3.3 Specialization Benchmarks

Specialization in LLM-based agents refers to activating domain-relevant reasoning through conditioning. We examine whether a role names an identity or binds a capability, and which structures the domain-specific reasoning encoded in the model. Using machine learning tasks from CatDB [16], we measure behavioral differences with fixed data and models.

Benchmark Structure and Conditioning Variants. The benchmark comprises five public datasets spanning regression, binary, and multiclass classification (Table 7), each evaluated under four conditioning strategies. *Role-based prompting* assigns a professional identity in the task description. *Planning-based conditioning* adds an intermediate step where the LLM generates a solution plan before code generation. *Expert-guided conditioning* injects methodological instructions reflecting data-science workflows. *Dataset-profile tooling* grants the agent a tool returning feature types, cardinality, and missingness, without prescribing a workflow. These variants isolate identity framing, reasoning structure, procedural guidance, and data access.

Evaluation Setup. Task structure, data loading, model execution, and metrics stay fixed. Only the conditioning strategy varies across runs. Each agent generates a data-centric ML pipeline, and we score the executed pipeline rather than the generated code. We report MAE for regression and precision, recall, and F1 for classification, so our results compare conditioning strategies on task performance rather than on output style.

3.4 Tool Use Benchmarks and Limitations

Tool use enables LLM-based agents to interact with external APIs and services beyond language generation. Recent benchmarks such as ToolBench and its deterministic variant StableToolBench [18, 51, 52] evaluate an agent's ability to select and invoke appropriate tools from large, heterogeneous registries.

Benchmark Structure and Scope. Figure 2 shows how StableToolBench spans multiple domains and interface complexities, including single-endpoint tools and multi-endpoint APIs [18].

[6]Annotated plans for the same question under each interface appear in the supplement.

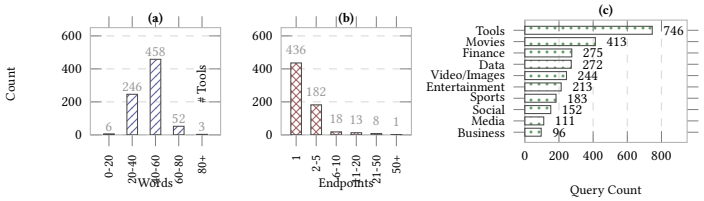


Figure 2: Characteristics of StableToolBench, showing the distributions of (a) query length, (b) number of APIs per tool, and (c) query counts across tool categories.

It stresses correct tool selection, parameter grounding, and multi-step interaction with closely related endpoints. MAFBench integrates it through a tool-use evaluation layer that preserves the original queries, server, and scoring logic. A pre-execution selection step bounds the tool subset per query before framework binding, so tool sets are identical across runs. We do not report quantitative tool-use results, because current frameworks delegate tool invocation entirely to the LLM without architectural control over selection. Tool-use evaluation is deferred until frameworks introduce explicit orchestration mechanisms.

3.5 Coordination and Scaling Benchmarks

Existing frameworks support agent execution and task decomposition, but they lack native multi-round peer-to-peer communication over arbitrary topologies. Interaction topology therefore dictates coordination rather than explicit framework mechanisms. We adopt AGENTSNET [17], which analyzes emergent collective behavior under constrained message passing.[7] It evaluates *local consistency* and *global alignment* under limited connectivity.

Benchmark Structure and Conditioning Variants. AGENTSNET includes five coordination tasks: *Coloring* (local conflict resolution), *Matching* (bilateral agreement), *VertexCover* (global resource minimization), *LeaderElection* (symmetry breaking), and *Consensus* (long-range information propagation) [17]. Agents interact only along edges of predefined graph families: *Small-World* [58], *Scale-Free* [35], and *Delaunay* [34]. This isolates how clustering, path length, and connectivity heterogeneity affect coordination dynamics.

Evaluation Setup. To reflect the interaction constraints of role-based and GABM frameworks, we implement a topology rewriting engine that derives sequential, hierarchical, and centralized structures from shared base graphs while preserving agent sets. The centralized structure mediates interaction through a communication hub that stands in for the GABM environment. We then execute identical coordination tasks under each pattern using consistent prompts, models, and task semantics, and sweep across tasks, graph families, and agent counts.

4 Evaluation

Each study varies a single framework-level dimension using our MAFBench pipeline, holding models, prompts, data, and execution semantics fixed. Frameworks provide mechanisms at two levels.

[7]Dataset: https://huggingface.co/datasets/disco-eth/AgentsNet; Code: https://github.com/floriangroetschla/AgentsNet

Table 8: MAFBench evaluation summary. Each row maps an architectural dimension to its benchmark, metrics, and research question (Section 2.3). One dimension varies per experiment while all others stay fixed.

Dimension	Benchmark	Metrics	RQ
Memory architecture	MemoryAgentBench [25]	Accuracy, F1, Recall@5	RQ1
Planning interface	GSM8K [12], CSQA [56], MATH [21]	Accuracy, failure rate, runtime	RQ2
Specialization	CatDB tasks [16]	Precision, Recall, F1	RQ3
Communication topology	AgentsNet [17]	Success, rounds, tokens, runtime	RQ4
Orchestration overhead	Trivial query (50 trials)	Latency, throughput, output size	RQ5

Per-agent mechanisms configure one agent: storage, planning interface, specialization, and the orchestration wrapper around a model call. Cross-agent mechanisms govern interaction: communication topology and coordination. We isolate the per-agent level first (Section 4.1), so cross-agent differences (Section 4.2) follow from coordination rather than confounded per-agent machinery. Table 8 maps each dimension to its benchmark and metrics. MemoryAgentBench aggregates heterogeneous subtask metrics as category-level means [25]. To manage cost, each configuration runs once. The planning interface is repeated five times on four models to bound variance (Section 4.3).

4.1 Single-Agent and Overhead Evaluation

We first examine how framework orchestration, memory, and planning affect execution cost and reasoning behavior.

4.1.1 Framework Overhead. We evaluate how orchestration architecture affects baseline execution cost, independent of memory, planning, tool use, and coordination. We fix the LLM (*gpt-4o-mini*), the query (*"What is 2+2?"*), prompts, and concurrency, and vary only the orchestration layer under minimal viable configurations: a raw *Direct LLM* baseline; graph- and role-based *LangGraph* [31], *OpenAgents* [63], *AutoGen* [62][8], *CrewAI* [26], *Agno* [3], and the *OpenAI SDK* [47]; the GABM framework *Concordia* [59]; a *minimal GABM skeleton* we implement to separate paradigm cost from implementation cost; and *OpenHands* [4] as an out-of-taxonomy reference. We run 50 trials and report median (p50) and tail (p95) latency, throughput, and output size.

Figure 3 compares each to the Direct LLM baseline (p50 495 ms, 7.0 req/s), which has the lowest latency and highest throughput. Most graph- and role-based frameworks add only a thin layer, from OpenAgents (527 ms) and LangGraph (548 ms) through CrewAI (741 ms), Agno (902 ms), AutoGen (964 ms), and the OpenAI SDK (1,275 ms). Concordia is the clear outlier at 30.6 s, roughly 60× the baseline, with throughput collapsing to 0.12 req/s and output reaching ∼100,000 characters against ∼15 elsewhere.

A trivial query issues identical model work everywhere, so cost differences originate in framework machinery. We decompose wall-clock time at the API boundary into LLM time and *framework-residual* time (Table 9). Residual stays under 21 ms and below 2.2% of latency for nearly all graph- and role-based frameworks, a light wrapper over one model call. Agno is the exception at 39.3%. Concordia inverts this, at 28.6 s and 91.5% of latency, with ∼3 calls per query. Prompt size does not explain it. OpenHands sends as large a prompt yet spends 12 ms residual. The cost is Concordia's

[8]AutoGen forked into AG2 (pyautogen) and an event-driven rewrite (`autogen-agentchat`). We evaluate `autogen-agentchat` 0.7.5 throughout. Per-experiment versions are pinned in the artifact.

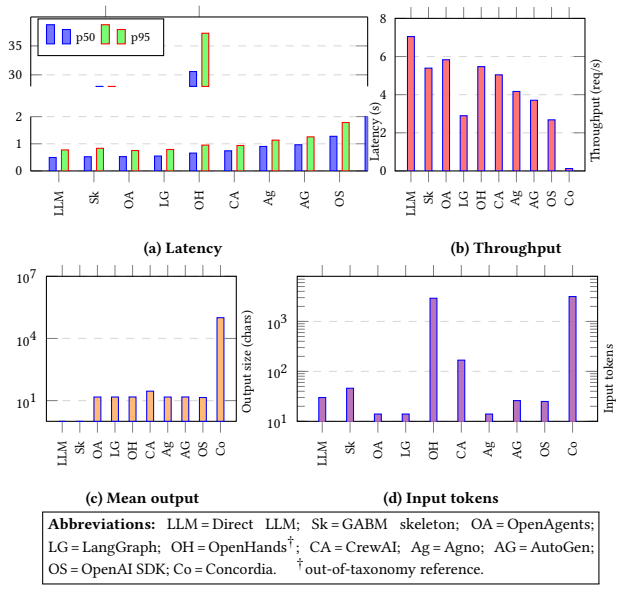


Figure 3: Orchestration overhead on a trivial task ("What is 2+2?") over 50 trials, ordered by p50 latency (Sk completed 46, OS 49). Graph- and role-based frameworks add modest overhead. The minimal GABM skeleton does too, so Concordia's orders-of-magnitude higher latency, lower throughput, and larger output are implementation, not paradigm, cost.

game-master mediation, which routes action resolution and narrative updates through the model, plus persistent loops and state serialization. The skeleton realizes the same paradigm without that machinery, and its residual is 13.7 ms (97.8% LLM), three orders of magnitude below Concordia. Environment mediation therefore costs milliseconds, and Concordia's overhead is implementation, not paradigm (RQ5). We report Concordia as deployed, because those numbers describe the real cost of using it. The light-wrapper result spans six frameworks and is paradigm-level. Concordia's is framework-level, and the skeleton supplies the GABM reference.

4.1.2 Memory Architecture Effects. We evaluate how memory architecture affects recall, in-session learning, long-range reasoning, knowledge revision, and runtime scalability. All frameworks are accessed through a common interface (Section 3.1) with identical chunking, ingestion order, and metrics, and all experiments use the same backend model (`gpt-oss-20b` on Groq[9]). Observed differences reflect memory execution semantics rather than model capability. We verify across capability with a Claude-Haiku-4.5 sensitivity row rather than a full frontier sweep, because the suite is long-context and repeating it at frontier pricing is prohibitive.[10]

Table 10 reveals three dominant memory architectures: *retrieval-centric* memory with persistent semantic stores, *accumulation-based* memory that replays recent interactions in the prompt, and *hybrid*

[9]Although `gpt-oss-20b` supports context windows up to 130K tokens, large accumulated contexts exceed tokens-per-minute limits (e.g., 250K TPM), making extreme accumulation impractical for evaluation.

[10]The suite costs roughly $300 at `gpt-oss-20b` pricing; frontier pricing raises this by one to two orders of magnitude.

Table 9: API-boundary cost decomposition on the trivial workload (50 trials, *gpt-4o-mini*), ordered by residual = total latency − LLM time. The skeleton against Concordia separates GABM *paradigm* cost from Concordia's *implementation* cost. †out-of-taxonomy reference.

Framework	p50 (ms)	Residual (ms)	Residual %	Calls	Input tok.
Direct LLM	495	–	–	1.0	30
OpenAgents	527	0.06	<0.1	1.0	14
LangGraph	548	1.7	0.2	1.0	14
AutoGen	964	5.9	0.6	1.0	26
OpenHands†	657	12.1	1.7	1.0	2,914
GABM skeleton	522	**13.7**	2.2	1.0	46
CrewAI	741	16.5	2.1	1.0	168
OpenAI SDK	1,275	20.8	1.5	1.0	25
Agno	902	365	**39.3**	1.0	14
Concordia	30,564	**28,643**	**91.5**	**3.0**	3,150

memory combining retrieval with bounded short-term accumulation. CrewAI is retrieval-centric. Agno and the OpenAI SDK are accumulation-based. LangGraph adopts a hybrid design. We evaluate LangGraph in retrieval-only and hybrid modes across context windows (W) and vary W for the OpenAI SDK to characterize accumulation scaling. Concordia and OpenAgents are excluded for narrative simulation semantics and absent native memory abstractions.

Accuracy and Reasoning Effects. Hybrid retrieval–accumulation performs best on Accurate Retrieval (AR). LangGraph (W=512) reaches *44.9*, above pure retrieval (*33.2*) and accumulation-based Agno (*19.1*). Retrieval dominates because it isolates relevant evidence from stored history, and accumulation degrades because unfiltered context dilutes signal (RQ1a). Accumulation-only memory rises to *32.3* for the OpenAI SDK at W=1024 and stays almost flat as context grows. Reliable factual recall depends on architectural retrieval, not larger context windows.

Long-Range Understanding (LRU) shows a similar pattern. Retrieval-only LangGraph achieves the highest score (*30.4*). Accumulation-based designs approach it only at very large context windows (OpenAI SDK at W=8192: *27.5*), and at higher cost. Hybrid designs perform worse on LRU, because short-term accumulation interferes with retrieved narratives and weakens temporal coherence (RQ1b).

In-Session Learning and Knowledge Revision. Test-Time Learning (TTL) shows the opposite trend. Accumulation-based designs improve steadily as history grows, with the OpenAI SDK rising from *1.2* at W=50 to *20.7* at W=8192. Hybrid designs outperform pure accumulation, and LangGraph (W=1024) achieves the highest TTL score (*28.9*) by anchoring learning to retrieved signals rather than prompt growth. Retrieval-only memory performs poorly, so adaptation requires accumulation but benefits from filtering (RQ1c).

Selective Forgetting (SF) remains largely unsupported. Retrieval-centric designs partially support single-hop forgetting, with retrieval-only LangGraph reaching *20.2*, but multi-hop forgetting stays below *5.0* across all architectures. Accumulation-based designs perform worse even at large context sizes, and hybrid designs collapse as outdated information persists in retrieval stores and accumulated context at once. Every evaluated framework-native architecture treats memory as append-only. Controlled knowledge revision requires explicit state update mechanisms, such as revise or remove, that no evaluated framework provides (RQ1d).

Effect of Model Capability. We evaluate LangGraph (Vector DB only) using Claude-Haiku-4.5 (Table 10, last row). AR stays stable (33.2 vs. 33.7), and the stronger LLM under retrieval-only design

Table 10: Memory architecture results on MemoryAgentBench. Three architectures are compared: retrieval-centric (CrewAI, LangGraph vector DB only), accumulation-based (Agno, OpenAI SDK), and hybrid (LangGraph with context window W). Scores are averaged per subtask across sessions. Category scores (AR, TTL, LRU, SF) are subtask means, and Overall is their average. Bold shaded cells indicate global maxima. Underlined values denote the best within each architecture.

Agent Framework	Accurate Retrieval (AR)					Test-Time Learning (TTL)			Long-Range Understanding (LRU)			Selective Forgetting (SF)			Overall
	SH-QA	MH-QA	LME(S*)	EventQA	Avg.	MCC	Recom.	Avg.	Summ.	DetQA	Avg.	FC-SH	FC-MH	Avg.	Score
***Retrieval-centric* Memories (LLM = gpt-oss-20b)**															
Crewai	14.0	22.0	3.0	48.7	21.9	5.6	12.2	8.9	0.0	19.7	9.9	26.2	0.8	13.5	13.5
LangGraph (Vector DB only)	14.0	24.0	31.3	63.3	33.2	22.8	0.0	11.4	20.0	40.8	**30.4**	36.0	4.5	**20.2**	**23.8**
***Accumulation-based* Memories (LLM = gpt-oss-20b)**															
Agno	12.0	19.0	2.0	43.2	19.1	0.6	15.1	7.8	0.0	18.3	9.2	27.0	0.8	13.9	12.5
OpenAI SDK ($W = 50$)	16.0	16.0	1.7	0.0	8.4	1.4	0.9	1.2	0.0	0.0	0.0	28.5	1.0	14.8	6.1
OpenAI SDK ($W = 512$)	8.0	11.0	1.0	44.1	16.0	0.6	17.2	8.9	0.0	28.2	14.1	28.7	0.8	14.8	13.4
OpenAI SDK ($W = 1024$)	40.0	33.0	7.7	48.5	32.3	3.6	22.1	12.8	0.0	25.4	12.7	0.0	0.0	0.0	14.5
OpenAI SDK ($W = 2048$)	32.0	32.0	18.3	49.9	33.1	10.0	20.9	15.5	0.0	33.8	16.9	0.2	0.0	0.1	16.4
OpenAI SDK ($W = 4096$)	36.0	35.0	14.0	48.5	33.4	16.2	22.7	19.4	0.0	29.6	14.8	0.5	0.0	0.2	17.0
OpenAI SDK ($W = 8192$)	34.0	34.0	23.0	44.7	33.9	34.0	7.3	20.7	10.0	45.1	27.5	0.8	0.0	0.4	20.6
***Hybrid Retrieval–Accumulation* Memories (LLM = gpt-oss-20b)**															
LangGraph ($W = 50$)	35.0	35.0	41.7	3.7	28.8	38.6	3.9	21.3	0.0	1.4	0.7	0.2	0.2	0.2	12.8
LangGraph ($W = 512$)	36.0	34.0	44.0	65.6	**44.9**	35.6	12.8	24.2	0.0	35.2	17.6	0.0	0.0	0.0	21.7
LangGraph ($W = 1024$)	34.0	35.0	41.3	64.2	43.6	38.0	19.8	**28.9**	0.0	40.8	20.4	0.0	0.2	0.1	23.3
LangGraph ($W = 2048$)	37.0	30.0	39.3	63.7	42.5	34.6	19.4	27.0	0.0	40.8	20.4	0.0	0.5	0.2	22.5
LangGraph ($W = 4096$)	33.0	34.0	36.7	60.4	41.0	32.2	14.6	23.4	0.0	29.6	14.8	0.0	0.0	0.0	19.8
LangGraph ($W = 8192$)	31.0	35.0	41.7	59.6	41.8	32.8	18.7	25.7	10.0	40.8	25.4	0.0	0.2	0.1	23.3
Model Sensitivity Check (LLM = Claude-Haiku-4.5)															
LangGraph (Vector DB only)	9.0	8.0	35.3	82.5	33.7	0.8	51.6	26.2	40.0	53.5	46.8	29.0	4.0	16.5	30.8

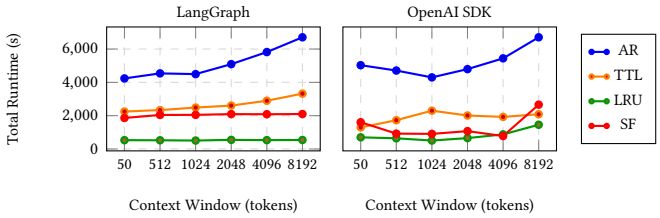


Figure 4: Runtime scaling with context window for LangGraph (hybrid retrieval–accumulation, left) and OpenAI SDK (pure accumulation, right) across four MemoryAgentBench competencies. Accumulation drives steep growth on high-query tasks (AR, TTL), while retrieval stays stable.

still does not reach the hybrid score of 44.9 with a weaker model, so architecture is the bottleneck (RQ1a). TTL improves, yet accumulation with a weaker LLM still outperforms retrieval-only with a stronger one, preserving the architectural ranking (RQ1c). LRU improves (30.4 to 46.8), so architecture and model capability contribute jointly. SF declines slightly, so knowledge revision remains unsupported regardless of model strength (RQ1d). Model capability affects absolute scores but does not alter architectural rankings or change the answer to any question.

Runtime and Scalability Effects. Figure 4 reports runtime against context window for LangGraph (hybrid) and the OpenAI SDK (pure accumulation). High-query tasks such as AR and TTL grow steeply under accumulation, where full histories are replayed for each query. LRU stays nearly flat for LangGraph because sessions carry few queries, and SF grows moderately. Retrieval avoids repeated processing, so scalability is governed by memory architecture rather than by the LLM's nominal context capacity. Retrieval-centric and accumulation-based results each span two independent frameworks and are architecture-level. The hybrid result rests on LangGraph alone and is framework-level.

4.1.3 Planning Interface Effects. We evaluate how the *planning interface* affects reasoning accuracy, robustness, and runtime, separating architectural constraints from the LLM's planning ability. The same model produces the plan under Crew-Plan and Direct-LLM-Plan, so the comparison isolates the interface. Using the MAFBench planning pipeline (Section 3.2), we fix the benchmarks (GSM8K [12], CSQA [56], MATH [21]), prompts, decoding settings, and scoring, and vary only the interface: *NoPlan* (direct answer), *Crew-Plan* (schema-constrained two-stage planning in CrewAI [26]), and *Direct-LLM-Plan* (free-form plan text injected before execution). For MATH we evaluate a 100-problem subset preserving the original complexity distribution.

Table 11 shows that schema-constrained planning reduces accuracy across datasets and models, but free-form planning preserves or improves it for weaker ones. Under Crew-Plan, Qwen-7B drops from *13.0%* to *3.4%* on GSM8K and from *69.9%* to *37.6%* on CSQA, and GPT-OSS-20B drops from *80.0%* to *48.0%* on MATH-100. Direct-LLM-Plan instead lifts Qwen-7B to *28.7%* on GSM8K and Llama-3.1-8B from *65.6%* to *71.9%*. Tables 12 and 13 identify the architectural cause. Crew-Plan produces large formatting failure rates, where the model violates the framework's required schema and the plan cannot be parsed even when the reasoning is correct. This reaches *84.7%* on GSM8K for Qwen-7B. Crew-Plan also adds an extra call with schema generation, validation, and parsing, raising runtime *7.4*× on GSM8K and *18.5*× on CSQA for GPT-OSS-20B, and exceeding *31*× for Llama-3.1-8B on CSQA. Direct-LLM-Plan yields *0.0%* formatting failures across all models, with smaller slowdowns (*0.8*×–*6.6*×).

Capability bounds the failure mechanism but not the cost. GPT-5.6-Luna, GPT-5.6-Terra, and Claude-Opus-5 never violate the schema, so parse failure is a weak-model phenomenon. Eliciting a plan does not help them either. Across all three benchmarks and both interfaces their accuracy remains almost flat, yet overhead persists at *2.5*×–*4.0*× for GPT-5.6 and *9.2*×–*15.0*× for Opus-5, whose longer plans make its absolute cost the largest measured. The interface therefore carries two independent costs. Parse failure scales inversely with capability and vanishes at the frontier. Plan injection and the extra call add runtime and token costs without benefit when tasks fit the model's context window. More complex tasks, such as software engineering, are future work. These results answer RQ2. Below the frontier, formatting failures rather than reasoning errors are the primary cause of accuracy loss under the schema-constrained interface. At the frontier the schema is satisfied and the residual cost is the plan itself. CrewAI is the only evaluated

Table 11: Accuracy (%) under three planning interfaces: ✗ no plan, ✓(Crew) schema-constrained, ✓(LLM) free-form. Schema-constrained planning reduces accuracy for weaker models through formatting failures. Both planning modes use the same model to generate the plan.

LLM	GSM8K [12]			CSQA [56]			MATH-100 [21]		
	✗	✓(crew)	✓(LLM)	✗	✓(crew)	✓(LLM)	✗	✓(crew)	✓(LLM)
Local Models									
GPT-OSS-20B [46]	94.6	91.6 ↓	87.6 ↓	81.4	81.2 ↓	82.1 ↑	80.0	48.0 ↓	76.0 ↓
Phi-4-14B [1]	93.3	86.7 ↓	83.5 ↓	81.7	82.0 ↑	74.6 ↓	72.0	50.0 ↓	69.0 ↓
Llama-3.1-8B [15]	65.6	60.3 ↓	71.9 ↑	70.1	65.4 ↓	65.4 ↓	25.0	24.0 ↓	37.0 ↑
Qwen-7B [7]	13.0	3.4 ↓	28.7 ↑	69.9	37.6 ↓	74.0 ↑	9.0	4.0 ↓	13.0 ↑
DeepSeek-7B [14]	27.0	18.0 ↓	23.5 ↓	46.3	31.2 ↓	46.1 ↓	9.0	3.0 ↓	4.0 ↓
Remote Models									
GPT-4.1 [48]	94.2	80.7 ↓	78.2 ↓	87.1	87.3 ↑	87.0 ↓	86.0	60.0 ↓	84.0 ↓
GPT-4o-Mini [48]	86.5	90.8 ↑	84.2 ↓	81.8	81.3 ↓	83.1 ↑	60.0	53.0 ↓	59.0 ↓
GPT-5.6-Luna	96.5	96.7 ↑	96.7 ↑	87.7	86.4 ↓	86.1 ↓	90.0	90.0 ↓	89.0 ↓
GPT-5.6-Terra	96.4	95.8 ↓	92.6 ↓	87.6	87.2 ↓	86.0 ↓	94.0	93.0 ↓	94.0 ↓
Claude-Opus-5	97.7	97.1 ↓	96.8 ↓	90.7	87.8 ↓	89.3 ↓	95.0	95.0 ↓	95.0 ↓

Table 12: Percentage of questions where the LLM output violated the required schema, under Crew-Plan and Direct-LLM-Plan. Crew-Plan failures reach 84.7% for weaker models. Direct-LLM-Plan produces zero violations across all models.

LLM	GSM8K		CSQA		MATH-100	
	✓(CrewAI)	✓(LLM)	✓(CrewAI)	✓(LLM)	✓(CrewAI)	✓(LLM)
Local Models						
GPT-OSS-20B	2.4%	0.0%	1.3%	0.0%	**49.0%**	0.0%
Phi-4-14B	6.1%	0.0%	0.5%	0.0%	21.0%	0.0%
Llama-3.1-8B	10.5%	0.0%	4.8%	0.0%	19.0%	0.0%
Qwen-7B	**84.7%**	0.0%	**41.0%**	0.0%	**70.0%**	0.0%
DeepSeek-7B	25.5%	0.0%	31.0%	0.0%	**40.0%**	0.0%
Remote Models						
GPT-4.1	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%
GPT-4o-Mini	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%
GPT-5.6-Luna	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%
GPT-5.6-Terra	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%
Claude-Opus-5	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%

Table 13: Runtime overhead relative to NoPlan under Crew-Plan and Direct-LLM-Plan. Crew-Plan increases runtime by up to 31×. Direct-LLM-Plan adds modest overhead (0.8×–6.6×). Lower values indicate less overhead. Overhead persists at frontier capability, and Opus-5 pays the largest, since it generates longer plans.

LLM	GSM8K		CSQA		MATH	
	Δ CrewAI	Δ LLM	Δ CrewAI	Δ LLM	Δ CrewAI	Δ LLM
Local Models						
GPT-OSS-20B	× 7.4	× 2.3	× 18.51	× 1.7	× 11.43	× 1.7
Phi-4-14B	× 3.5	× 1.4	× 4.1	× 1.6	× 3.0	× 1.7
Llama-3.1-8B	× 6.3	× 1.3	× 31.34	× 4.2	× 4.4	× 1.3
Qwen-7B	× 2.2	× 2.5	× 18.16	× 0.9	× 2.9	× 1.8
DeepSeek-7B	× 3.1	× 2.6	× 13.29	× 6.6	× 2.8	× 1.2
Remote Models						
GPT-4.1	× 2.4	× 1.5	× 2.9	× 1.1	× 1.2	× 1.1
GPT-4o-Mini	× 1.5	× 1.4	× 4.8	× 2.3	× 1.8	× 1.4
GPT-5.6-Luna	× 2.6	× 1.6	× 4.0	× 1.5	× 2.5	× 1.2
GPT-5.6-Terra	× 2.9	× 1.5	× 3.2	× 0.8	× 2.5	× 1.3
Claude-Opus-5	× 15.00	× 2.7	× 14.57	× 2.6	× 9.2	× 2.3

framework with a runtime planning interface, so this is framework-level. It holds across ten models spanning two orders of magnitude in capability, so it is not model-specific.

4.1.4 Agent Specialization. We evaluate how specialization mechanisms shape domain-specific reasoning behavior. A role can name an identity or bind a capability. Using the unified specialization pipeline (Section 3.3), we fix the LLM backend (GPT-4o-mini), datasets from CatDB [16], prompts, task structure, and evaluation metrics, and vary only the conditioning strategy across the four conditions of Table 14. A professional label and a label with self-generated planning supply identity, binding nothing to the role.

Table 14: Specialization under four conditioning strategies (GPT-4o-mini). Identity conditioning gives no gain over *No Role*. Capability conditioning helps only when it supplies procedure, so the dataset-profile tool does not reproduce the expert-guided result.

Category	Regres.	Binary Class			Multiclass								
Agent Role	Utility	WiFi			EU-IT			Yelp			Volkert		
	MAE	P	R	F1	P	R	F1	P	R	F1	P	R	F1
Role-as-identity: role-based prompting													
No Role	0.067	40.2	45.0	37.3	32.0	38.0	35.0	41.9	44.1	41.9	67.3	66.5	65.3
Data Scientist	0.067	40.2	45.0	37.3	32.3	37.9	34.8	41.9	44.1	41.9	67.3	66.5	65.3
Researcher	0.067	40.2	45.0	37.3	32.3	37.9	34.8	42.1	44.0	41.9	67.3	67.0	65.8
Data Analyst	0.067	40.2	45.0	37.3	33.3	37.9	34.8	42.1	44.0	41.9	67.0	67.0	65.0
Engineer	0.067	40.2	45.0	37.3	32.0	38.0	35.0	41.9	44.1	41.9	67.3	66.5	65.3
Role label with self-generated plan													
No Role	0.067	40.0	45.0	37.0	32.0	38.0	35.0	42.0	44.1	42.0	67.0	67.0	65.0
Data Scientist	0.067	40.0	45.0	37.0	32.0	38.0	35.0	42.0	44.1	42.0	67.0	67.0	66.0
Researcher	0.067	40.0	45.0	37.0	32.0	38.0	35.0	42.0	44.0	42.0	67.0	67.0	65.0
Data Analyst	0.067	40.0	45.0	37.0	32.0	38.0	35.0	42.0	44.0	42.0	67.0	67.0	65.0
Engineer	0.067	40.0	45.0	37.0	32.0	38.0	35.0	42.0	44.0	42.0	67.0	67.0	65.0
Role-as-capability: expert-guided procedure													
No Role	0.067	53.0	90.0	67.0	33.6	41.4	36.8	96.0	96.0	96.0	94.8	94.7	94.6
Data Scientist	0.068	57.1	80.0	66.7	37.3	44.8	40.3	100.0	100.0	100.0	95.0	95.0	95.0
Researcher	0.068	53.0	90.0	67.0	37.3	41.4	38.8	100.0	100.0	100.0	95.0	94.9	94.9
Data Analyst	0.067	52.9	90.0	66.7	37.3	41.4	38.8	100.0	100.0	100.0	94.7	94.6	94.5
Engineer	0.068	50.0	90.0	64.3	37.3	41.4	38.8	100.0	100.0	100.0	94.8	94.7	94.6
Role-as-capability: dataset-profile tool													
No Role	0.0682	62.2	65.0	62.0	36.8	28.3	25.6	45.6	46.0	40.8	66.5	66.1	64.6
Data Scientist	0.07	62.2	65.0	62.0	24.9	35.6	24.9	44.5	45.8	41.0	67.3	66.9	65.7
Researcher	0.065	73.3	84.6	**78.6**	25.7	35.6	25.5	44.2	45.7	41.0	66.5	66.1	64.6
Data Analyst	0.049	63.9	65.0	64.3	33.9	39.2	32.7	45.5	45.6	39.1	67.5	66.7	65.4
Engineer	0.068	68.8	84.6	75.9	17.5	30.8	**18.7**	46.1	45.5	38.9	66.5	66.1	64.6

Expert-guided instructions and the dataset-profile tool bind a capability. The tool grants data access with no prescribed workflow, which isolates it from procedural knowledge.

Identity conditioning gives nearly identical performance across all professional labels, with no gain over the *No Role* baseline, and self-generated planning stabilizes behavior without improving accuracy. Labels fail because they do not constrain the solution workflow. Expert-guided procedure does, lifting F1 above 94 on Yelp and Volkert against roughly 42 and 65 under labels.[11] The dataset-profile tool separates data access from procedure. It helps where the obstacle is knowing the data, raising WiFi F1 from 37.3 to 78.6, above the expert-guided range. It does not help where the obstacle is method. Yelp and Volkert stay near their label values, and EU-IT falls to 18.7 as the profile pushes the model toward features it cannot exploit. Bound capability therefore aids specialization only when it constrains the workflow (RQ3). Conditioning is applied in the prompt, independent of any framework, so this result is not tied to one implementation.

4.2 Multi-Agent Evaluation

We evaluate how communication architecture affects coordination success, scalability, and execution cost. Using the MAFBench coordination pipeline, we fix the LLM, task semantics, prompts, stopping criteria, and metrics, and vary only the topology through controlled graph structures. Agents communicate only with direct neighbors, so what each observes follows from the topology alone. A globally readable shared state would make every structure equally dense and remove the variable under study. This isolates the effect of communication architecture as agent populations grow.

4.2.1 Communication, Coordination, and Scaling. We evaluate how communication topology shapes coordination outcomes under constrained message passing using AGENTSNET [17]. All experiments

[11] Example prompts are provided in the supplementary materials.

Table 15: Coordination scalability across topology classes on AGENTSNET. Each entry reports performance for $n = 4, 8, 16, 50, 100$ agents. All (*) simulates GABM-style centralized communication using a fully connected graph. Runs with rounds > 40 are excluded for sequential pipelines at large n. Only fully connected topology achieves consistent consensus, answering RQ4.

		Graph-based (Small-World)					Graph-based (Scale-Free)					Graph-based (Delaunay)					Role-based (Sequential)					Role-based (Hierarchical)					GABM-based (All)*				
Task	**Metric / n**	4	8	16	50	100	4	8	16	50	100	4	8	16	50	100	4	8	16	50	100	4	8	16	50	100	4	8	16	50	100
Coloring	Success Rate	0.83	0.94	0.91	0.92	0.98	1.0	1.0	0.96	0.96	0.97	0.8	1.0	0.77	0.93	0.81	1.0	1.0	0.93	-	-	1.0	0.86	1.0	0.98	0.98	-	-	-	-	-
	Rounds to Converge	8	8	8	11	15	8	8	8	11	11	8	8	8	13	19	8	8	8	>40	>40	8	8	8	13	15	-	-	-	-	-
	Token Cost ($\times 10^3$)	125	274	481	2,993	10,431	109	233	508	2,906	5,699	116	270	555	4,362	18,108	95	205	444	$$	$$	104	204	420	3,115	8,102	-	-	-	-	-
	Runtime (sec)	72	108	111	464	929	66	108	117	378	652	118	120	158	507	1209	72	97	118	-	-	85	101	150	432	865	-	-	-	-	-
Matching	Success Rate	0.5	0.75	0.88	0.68	0.68	0.5	1.0	0.94	0.68	0.68	1.0	0.75	1.0	0.64	0.0	1.0	0.5	1.0	-	-	1.0	0.88	1.0	0.80	0.66	1.0	1.0	0.75	0.24	0.42
	Rounds to Converge	8	8	8	11	15	8	8	8	11	11	8	8	8	13	19	8	8	8	>40	>40	8	8	8	13	15	8	8	8	3	3
	Token Cost ($\times 10^3$)	112	249	538	2,888	9,847	102	217	482	2,684	5,274	107	268	547	4,265	18,283	90	198	423	$$	$$	94	203	395	2,951	7,827	107	340	829	1,124	3,016
	Runtime (sec)	111	123	141	311	910	88	95	120	306	577	77	81	109	462	1,400	67	90	124	-	-	83	88	112	412	760	74	134	196	182	621
VertexCover	Success Rate	0.83	0.79	0.99	0.88	0.81	0.0	0.29	0.64	0.80	0.85	1.0	0.0	0.95	0.84	0.80	0.67	0.57	0.73	-	-	0	0.57	0.3	0.57	0.72	-	-	-	-	-
	Rounds to Converge	8	8	8	11	15	8	8	8	11	11	8	8	8	13	18	8	8	8	>40	>40	8	8	8	13	15	-	-	-	-	-
	Token Cost ($\times 10^3$)	124	350	1,210	3,391	11,536	122	259	569	3,185	6,455	131	307	635	5,066	20,335	110	264	489	$$	$$	113	226	463	3,359	9,109	-	-	-	-	-
	Runtime (sec)	91	118	321	446	1,057	96	111	133	341	670	88	116	130	621	1,516	80	172	99	-	-	94	107	126	396	912	-	-	-	-	-
LeaderElection	Success	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗	✓	✓	✓	✓	✗	✗	✓	-	-	✗	✓	✗	✗	✗	✓	✗	✗	✗	✗
	Rounds to Converge	3	7	7	11	15	5	7	9	11	11	5	5	9	13	19	7	15	31	>40	>40	5	7	9	13	15	3	3	3	3	3
	Token Cost ($\times 10^3$)	29	220	438	2,985	10,371	51	182	608	2,929	5,858	54	124	707	4,595	18,784	90	645	5,216	$$	$$	48	168	512	3,179	8,168	29	72	194	1,229	2,784
	Runtime (sec)	35	102	157	327	867	50	73	125	412	615	41	64	168	483	1,463	86	207	666	-	-	54	77	107	444	866	40	47	61	246	515
Consensus	Success	✓	✗	✓	✗	✗	✓	✓	✓	✗	✗	✓	✗	✓	✗	✗	✗	✓	✗	-	-	✓	✗	✗	✗	✗	✓	✓	✓	✓	✓
	Rounds to Converge	3	7	7	11	15	5	7	9	11	11	5	5	9	13	19	7	15	31	>40	>40	5	7	9	13	15	3	3	3	3	3
	Token Cost ($\times 10^3$)	26	211	379	2,892	10,664	51	170	594	2,793	5,734	52	122	629	4,222	18,283	76	620	5,183	$$	$$	49	163	505	3,061	8,336	24	61	175	1,151	2,611
	Runtime (sec)	22	101	86	378	1,010	54	77	134	372	733	48	62	145	472	1,400	71	201	526	-	-	60	116	162	402	959	26	32	50	228	922

fix the model backend, task definitions, prompts, execution protocol, and round budgets through the unified MAFBench infrastructure, and vary only the interaction structure (Section 3.5). We compare graph-based topologies (small-world [58], scale-free [35], and Delaunay [34]), role-based pipelines (sequential and hierarchical), and a fully connected structure standing in for GABM centralization. Network size varies over $n \in \{4, 8, 16, 50, 100\}$. We report task success, rounds to convergence, token usage, and runtime (Table 15). Bounded tasks run a minimum of eight rounds. Success is binary for the global agreement tasks *LeaderElection* and *Consensus*.[12] The maximum follows $T = 2D + 1$, where D is the graph diameter, so information has enough rounds to propagate.[13] Under direct message passing, connectivity and visibility coincide, so full connectivity gives global visibility, and is the only structure reaching consensus. A shared environment decouples the two, because a structurally sparse star may still expose global state to every agent. Coordination under shared-state environments is future work.

Local tasks tolerate sparse structure. *Coloring* succeeds across every graph family, and *Matching* holds from 0.5 for all graph-based families before n=50, because each agent needs only its neighbors. Topology dependence is stronger on tasks that need global information. *VertexCover* minimizes a global objective from local decisions, and scale-free graphs dominate at large scale in both success and efficiency. For *LeaderElection*, small-world and scale-free structures fail at every size, and Delaunay graphs succeed only at moderate and large scale at high coordination cost. Role-based hierarchies and centralized interaction succeed only at very small scale, then degrade rapidly. For *Consensus*, the fully connected topology is the only structure that succeeds consistently, converging in three rounds independent of network size. Every graph-based topology fails despite substantially higher runtime and token expenditure. Consensus requires every agent to receive information from every other agent within bounded rounds, and sparse graphs create fragmented clusters that cannot merge. This answers RQ4.[14]

Coordination scalability is therefore an architectural property of communication structure. Local coordination benefits from sparse topologies that preserve neighborhood structure. No single topology performs well across all tasks, so communication architecture should match the information flow the task requires. Topologies are instantiated by MAFBench's rewriting engine rather than by vendor frameworks, so these results are structure-level and carry no framework-implementation confound.

4.3 Scope and Reflection

Each experiment fixes the LLM and task to isolate one architectural dimension, so results are not verified across all model families, scales, or workloads. We partially address this with multiple LLM backends. Concordia is one GABM implementation, so its absolute numbers do not characterize the paradigm. The minimal skeleton supplies the paradigm-level reference point. LLM outputs vary across identical runs, so we repeat the planning-interface experiment on MATH-100 five times each for GPT-5.6-Terra, GPT-5.6-Luna, GPT-OSS-20B, and Llama-3.1-8B, spanning the capability range.[15] Accuracy varies by at most 4.1 points and schema-violation rate by at most 3.3, small relative to the differences the paper attributes to the interface, so single-run reporting preserves the architectural rankings. Aggregation absorbs further noise, because every score is a mean over 100 questions, and memory and coordination scores average over subtasks and graph instances too. All five research questions are answered (Section 2.3). Retrieval dominates multi-hop retrieval and in-session learning, long-range understanding benefits from both retrieval architecture and model capability, and knowledge revision fails across all evaluated framework-native architectures and both LLMs (RQ1). Schema-constrained planning reduces accuracy through formatting failures below the frontier,

[12] Detailed configurations, success definitions for all tasks, and metric formulations are provided in the supplementary materials.
[13] The graph diameter D is the maximum shortest-path distance between any pair of nodes in the communication graph.
[14] Consensus dynamics across all topologies and network sizes are visualized in the supplementary materials, showing persistent fragmentation in sparse graphs.
[15] Per-model means and population standard deviations are in the supplementary materials.

above which the plan itself is the cost (RQ2). Role labels do not activate domain reasoning, and bound capability helps only when it supplies procedure (RQ3). Sparse topologies fail on global agreement (RQ4). Environment mediation costs milliseconds, so Concordia's order-of-magnitude overhead is implementation cost (RQ5). These effects hold across the *evaluated* frameworks, datasets, and LLMs, converging on one theme: architectural design, not model intelligence alone, governs multi-agent behavior.

5 Multi-Agent Design Principles

This section distills results from Section 4 into actionable design principles for multi-agent LLM systems. Each principle is grounded in controlled empirical evidence and states both where it applies and where it does not.

Principle 1: Orchestration overhead is the dominant scalability constraint. *Judge orchestration by its execution structure, not by its paradigm.* Framework execution structure governs latency and throughput even for trivial workloads (Section 4.1.1, RQ5). Direct LLM calls incur minimal cost. Graph- and role-based frameworks add a thin wrapper over the model call, with Agno the exception. Concordia's GABM execution produces orders-of-magnitude higher latency from game-master mediation, persistent loops, and state serialization. A minimal implementation of the same paradigm does not. These effects arise from orchestration semantics, not task complexity. *Scope:* this principle applies to latency-sensitive and throughput-sensitive production systems such as high-volume document processing. For tasks that require multi-round deliberation such as social simulation, deeper orchestration may be justified despite overhead. Measure the framework rather than assuming its paradigm's cost. Two implementations of one paradigm differed by three orders of magnitude here. The key decision is whether the task requires coordination across rounds or decomposes into independent single-agent calls.

Principle 2: Framework-native memory must be architected around task semantics, not context size. *Choose memory architecture from task needs, and never rely on larger context windows.* Architectural design governs memory behavior rather than available context (Section 4.1.2, RQ1). Retrieval-first architectures dominate factual recall and long-range reasoning by isolating relevant evidence. Accumulation-based memory degrades as unfiltered history expands, even under large context budgets. Hybrid designs achieve the strongest in-session learning by filtering accumulated signals through retrieval. No single architecture performs well across all competencies. Selective Forgetting fails across all evaluated framework-native architectures, which treat memory as append-only and lack explicit editing mechanisms. *Scope:* use retrieval-first memory for knowledge-intensive tasks such as multi-hop QA. Use bounded accumulation for session-oriented tasks such as tutoring. Never replace architectural memory control with larger context windows. Applications requiring controlled knowledge revision, such as correcting outdated facts, need primitives no evaluated framework provides natively.

Principle 3: Planning interfaces should be permissive, and planning itself optional. *Elicit a plan only when the model needs one, and keep the interface permissive.* The interface and the plan carry separate costs (Section 4.1.3, RQ2). Formatting failures, not reasoning errors, dominate accuracy loss for weaker models, and free-form planning removes them at modest runtime cost. Frontier models never violate the schema, so that cost vanishes. The plan still does not pay. Accuracy remains almost flat under either interface, and every model pays the extra call. *Scope:* skip the plan where the task fits the model's context and it plans internally. Enforce structure only when downstream components require machine-readable output, because recovering from a parse failure costs more than post-processing a loose plan.

Principle 4: Specialization should be procedural, and role labels secondary. *Encode expertise as workflows, not as role names.* Assigning professional roles does not activate domain-specific reasoning (Section 4.1.4, RQ3), because labels do not constrain how the model processes data. Self-generated planning stabilizes behavior without accuracy gains. Expert-guided conditioning delivers large improvements by imposing explicit workflows. Binding a capability is not sufficient either. A dataset-profile tool grants data access without procedure, and helps only where the obstacle is knowing the data. *Scope:* implement specialization through reusable procedural templates, such as data cleaning pipelines. Supply tools when the agent lacks information and procedure when it lacks method, because the two are not interchangeable. Role labels may still support routing or delegation. These findings use GPT-4o-mini across all five CatDB datasets, but the effect may differ for models with stronger instruction following.

Principle 5: Coordination is governed by communication topology, not by agent intelligence. *Design communication structure explicitly, because more interaction cannot fix bad topology.* Communication topology rather than interaction budget determines coordination success and scalability (Section 4.2.1, RQ4). For local coordination tasks, scale-free graphs offer the best success-cost trade-offs through efficient information routing. Sparse graphs with long paths fail on global agreement even under large round and token budgets. Only fully connected topology achieves consistent consensus, because every agent must receive information from every other agent within bounded rounds. *Scope:* full connectivity enables global agreement but incurs quadratic communication cost. For local coordination tasks such as distributed conflict resolution, where agents need only neighborhood information, sparse topologies provide better cost-success trade-offs. For tasks requiring system-wide consensus such as leader election, fully connected or near-complete topologies are necessary. Select communication architecture from whether the task requires local or global information flow.

Principle 6: System interfaces dominate multi-agent behavior. *Engineer robustness and scalability through interfaces, not prompts.* Across all five dimensions, performance limits arise from framework execution semantics and interface design rather than task difficulty or model capability. Orchestration determines latency and throughput, memory architecture controls recall and scalability, planning interfaces govern accuracy, specialization depends on procedural structure, and topology determines coordination success. *Scope:* invest engineering effort in interface design first. Select orchestration depth, memory access mode, planning format, and communication topology before tuning prompts or selecting models. Architectural bottlenecks persist across model generations, and additional rounds, tokens, or prompt refinement cannot compensate for them.

6 Future Directions

This section outlines research directions motivated by the architectural limitations identified in our evaluation. We identify what is missing and state the open research question.

Extending MAFBench. MAFBench currently evaluates five architectural dimensions under single-model configurations. Three extensions would broaden its scope. First, built-in cross-model sweeps with multi-run statistical reporting would evaluate every dimension across multiple LLM backends and strengthen reproducibility. Second, industry-representative workload profiles, such as long-running sessions, high-concurrency pipelines, and mixed-task workflows, would complement the current academic benchmarks. Third, controlled tool-use benchmarks would isolate framework-level tool orchestration from model-level tool calling once frameworks expose architectural control over tool selection. Repeated LLM-based evaluation at scale remains a practical barrier that requires cost-aware scheduling and selective replication.

Memory Editing and Lifecycle Management. Every evaluated framework-native architecture treats memory as append-only (Section 4.1.2), providing no controlled revision or dependency-aware deletion. Adaptive memory systems and plugin layers [30, 64] update and refine stored knowledge, but lack mechanisms for safe removal. Future memory architectures should support explicit state updates, namely revise, remove, and invalidate, with dependency tracking across stored facts. A concrete solution requires versioned fact stores with multi-version concurrency control, write-ahead logging for reversible updates, and conflict resolution for concurrent agent writes. Memory stores in multi-agent systems therefore need the transactional semantics, versioning, and consistency guarantees that databases already provide.

Multi-Agent Frameworks for Data-Intensive Applications. Multi-agent systems increasingly serve data-intensive applications, such as knowledge graph question answering, ETL pipeline orchestration, data integration, and conversational analytics. The trade-offs exposed by MAFBench directly affect these workloads. Memory scalability governs how agents manage large knowledge bases, coordination topology determines how agents distribute data processing, and orchestration overhead constrains real-time pipelines. Current practice requires manual architecture selection per application, which does not scale. The long-term goal is agentic systems that autonomously construct multi-agent architectures to automate complex industry roles, such as data engineering and financial analysis. This treats multi-agent system construction as a compilation problem that translates high-level task specifications into concrete orchestration, memory, planning, specialization, and coordination configurations. Section 5 supplies the optimization objectives and MAFBench the evaluation infrastructure, so connecting them into an automated pipeline is an open systems problem at the intersection of multi-agent design and data management.

Safety and Security. Our taxonomy identifies three dimensions with direct safety implications. Communication structure C affects adversarial information propagation. Storage design $\mathcal{S}$ influences persistence of sensitive data in agent memory. Orchestration O governs execution of external actions without authorization. Future benchmarks should measure concrete security properties under different configurations: prompt injection propagation rate, memory poisoning persistence, and unauthorized tool execution rate.

7 Related Work

Existing benchmarks focus on evaluating individual model or agent capabilities, but do not support controlled experimental analysis of the system-level factors that govern multi-agent framework performance. Works such as HELM [37] and Chatbot Arena [11] compare language models under standardized evaluation protocols, ensuring that observed differences are attributable to the model while treating the execution framework as fixed. This paradigm extends to agent settings in benchmarks such as AgentBench [39], WebArena [69], and SWE-bench [28], which evaluate agents in realistic and interactive environments but keep the environment, execution pipeline, and framework design unchanged across comparisons. As a result, these benchmarks measure capability improvements without enabling analysis of which architectural factors dominate performance or how design choices impact system behavior.

A second line of work explores multi-agent architectures more directly, but lacks controlled isolation of framework-level variables. BOLAA [40] evaluates multiple agent architectures combined with different LLM backbones, jointly varying both factors and confounding their effects. MetaGPT [22] proposes a structured multi-agent pipeline based on standardized operating procedures, but evaluates a single fixed design without varying architectural dimensions. Consequently, prior work does not provide controlled experiments to identify the principles governing multi-agent system design or to quantify the impact of individual architectural decisions. MAFBench addresses this gap by fixing the underlying LLM and task while varying architectural dimensions, enabling controlled attribution of performance differences to framework design choices such as memory, planning interfaces, and communication topology.

8 Conclusion

We introduce an architectural taxonomy across five dimensions (orchestration, memory, planning, specialization, and communication topology), yielding five research questions, all answered. MAFBench enables controlled, architecture-level comparison under fixed models and tasks. Our study across nine evaluated frameworks shows that architectural design, not model intelligence alone, governs performance. Rigid planning schemas make up to 84.7% of questions unparseable, so formatting failures rather than reasoning errors dominate accuracy loss. Only fully connected topology achieves consensus at scale. Role labels activate no domain reasoning, but bound procedure lifts F1 above 94. Environment mediation costs milliseconds in a minimal implementation, isolating a production framework's order-of-magnitude overhead as implementation cost. Memory architecture governs recall and scalability independent of context window size, and no evaluated framework natively supports knowledge revision. These findings parallel the trade-offs that drove database systems toward modular designs, motivating research on adaptive architectures, principled coordination, and automated compilation of agentic systems.

References

[1] Marah I. Abdin, Jyoti Aneja, Harkirat S. Behl, Sebastien Bubeck, et al. 2024. Phi-4 Technical Report. *Computing Research Repository (CoRR)* abs/2412.08905 (2024). https://doi.org/10.48550/arXiv.2412.08905

[2] Dennis Abts, John Kim, Garrin Kimmell, Matthew Boyd, Kris Kang, Sahil Parmar, Andrew Ling, Andrew Bitar, Ibrahim Ahmed, and Jonathan Ross. 2022. The Groq Software-Defined Scale-Out Tensor Streaming Multiprocessor: From Chips to Systems Architectural Overview. In *Proceedings of the IEEE Hot Chips 34 Symposium (HCS)*. 1–69. https://doi.org/10.1109/HCS55958.2022.9895630

[3] Agno AGI. 2025. Agno: A Multi-Agent Framework, Runtime, and Control Plane Built for Speed, Privacy, and Scale. https://github.com/agno-agi/agno. Accessed: January 2026.

[4] All Hands AI. 2026. OpenHands: An Open Platform for AI-Driven Software Development Agents. https://docs.openhands.dev/ Accessed: August 2026.

[5] Anthropic. 2026. Claude Code: An Agentic Coding Tool for the Terminal and IDE. https://docs.claude.com/en/docs/claude-code/overview Accessed: August 2026.

[6] Anysphere. 2026. Cursor: An AI Code Editor. https://docs.cursor.com/ Accessed: August 2026.

[7] Jinze Bai, Shuai Bai, Shusheng Yang, Shijie Wang, et al. 2024. Qwen-VL: A Versatile Vision-Language Model for Understanding, Localization, Text Reading, and Beyond. https://openreview.net/forum?id=qrGjFJVl3m

[8] Xiaohe Bo, Zeyu Zhang, Quanyu Dai, Xueyang Feng, Lei Wang, Rui Li, Xu Chen, and Ji-Rong Wen. 2024. Reflective Multi-Agent Collaboration Based on Large Language Models. In *Proceedings of the Thirty-Seventh Conference on Neural Information Processing Systems (NeurIPS)*, Vol. 37. NeurIPS Foundation, Vancouver, Canada, 138595–138631. https://proceedings.neurips.cc/paper_files/paper/2024/file/fa54b0edce5eef0bb07654e8ee800cb4-Paper-Conference.pdf

[9] Inigo Casanueva, Tadas Temcinas, Daniela Gerz, Matthew Henderson, and Ivan Vulic. 2020. Efficient Intent Detection with Dual Sentence Encoders. In *Proceedings of the 2nd Workshop on Natural Language Processing for Conversational AI (NLP4ConvAI)*. 38–45. https://aclanthology.org/2020.nlp4convai-1.5/

[10] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building Production-Ready AI Agents With Scalable Long-Term Memory. *arXiv Preprint* (2025). https://arxiv.org/pdf/2504.19413

[11] Wei-Lin Chiang, Lianmin Zheng, Ying Sheng, Anastasios Nikolas Angelopoulos, Tianle Li, Dacheng Li, Banghua Zhu, Hao Zhang, Michael Jordan, Joseph E Gonzalez, et al. 2024. Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference. *Proc. International Conference on Machine Learning (ICML)* (2024). https://openreview.net/pdf?id=3MW8GKNyzI

[12] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, et al. 2021. Training Verifiers to Solve Math Word Problems. *Computing Research Repository (CoRR)* abs/2110.14168 (2021). https://arxiv.org/abs/2110.14168

[13] OpenClaw Contributors. 2026. OpenClaw: An Open-Source Personal AI Agent. https://github.com/openclaw/openclaw Accessed: August 2026.

[14] DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, et al. 2024. DeepSeek-V3 Technical Report. *Computing Research Repository (CoRR)* abs/2412.19437 (2024). https://doi.org/10.48550/arXiv.2412.19437

[15] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, et al. 2024. The Llama 3 Herd of Models. *Computing Research Repository (CoRR)* abs/2407.21783 (2024). https://doi.org/10.48550/arXiv.2407.21783

[16] Saeed Fathollahzadeh, Essam Mansour, and Matthias Boehm. 2025. CatDB: Data-Catalog-Guided, LLM-Based Generation of Data-Centric ML Pipelines. *Proceedings of the VLDB Endowment (PVLDB)* 18, 8 (2025), 2639–2652. https://www.vldb.org/pvldb/vol18/p2639-fathollahzadeh.pdf

[17] Florian Grotschla, Luis Muller, Jan Tonshoff, Mikhail Galkin, and Bryan Perozzi. 2025. AgentsNet: Coordination and Collaborative Reasoning in Multi-Agent LLMs. *Computing Research Repository (CoRR)* abs/2507.08616 (2025). https://arxiv.org/abs/2507.08616

[18] Zhicheng Guo, Sijie Cheng, Hao Wang, Shihao Liang, Yujia Qin, Peng Li, Zhiyuan Liu, Maosong Sun, and Yang Liu. 2024. StableToolBench: Towards Stable Large-Scale Benchmarking on Tool Learning of Large Language Models. In *Proceedings of the Findings of the Association for Computational Linguistics (ACL Findings)*. 11143–11156. https://doi.org/10.18653/v1/2024.findings-acl.664

[19] Junda He, Christoph Treude, and David Lo. 2025. LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision, and the Road Ahead. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 34, 5 (2025), Article 124. https://doi.org/10.1145/3712003

[20] Zhankui He, Zhouhang Xie, Rahul Jha, Harald Steck, Dawen Liang, Yesu Feng, Bodhisattwa Prasad Majumder, Nathan Kallus, and Julian McAuley. 2023. Large Language Models as Zero-Shot Conversational Recommenders. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management (CIKM)*. 720–730. https://doi.org/10.1145/3583780.3614949

[21] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring Mathematical Problem Solving with the MATH Dataset. In *Proceedings of the Thirty-Fifth Conference on Neural Information Processing Systems, Datasets and Benchmarks Track (NeurIPS)*. https://openreview.net/forum?id=7Bywt2mQsCe

[22] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. 2023. MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework. *Proc. International Conference on Learning Representations (ICLR)* (2023). https://openreview.net/pdf?id=VtmBAGCN7o

[23] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, and Boris Ginsburg. 2024. RULER: What's the Real Context Size of Your Long-Context Language Models?. In *Proceedings of the First Conference on Language Modeling (COLM)*. https://openreview.net/forum?id=kIoBbc76Sy

[24] Chuanrui Hu, Xingze Gao, Zuyi Zhou, Dannong Xu, and et al. 2026. EverMemOS: A Self-Organizing Memory Operating System for Structured Long-Horizon Reasoning. *arXiv preprint arXiv:2601.02163* (2026). https://arxiv.org/abs/2601.02163

[25] Yuanzhe Hu, Yu Wang, and Julian McAuley. 2025. Evaluating Memory in LLM Agents via Incremental Multi-Turn Interactions. In *Proceedings of the International Conference on Machine Learning Workshop on Long-Context Foundation Models (ICML Workshop)*. https://openreview.net/forum?id=ZgQ0t3zYTQ

[26] CrewAI Inc. 2025. CrewAI: A Framework for Building Role-Based Multi-Agent Systems with LLMs. https://www.crewai.com/ Accessed: January 2026.

[27] Cline Bot Inc. 2026. Cline: An Autonomous Coding Agent for the IDE. https://github.com/cline/cline Accessed: August 2026.

[28] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *Proc. International Conference on Learning Representations (ICLR)* (2024). https://openreview.net/pdf?id=VTF8yNQM66

[29] Yiqiao Jin, Qinlin Zhao, Yiyang Wang, Hao Chen, Kaijie Zhu, Yijia Xiao, and Jindong Wang. 2024. AgentReview: Exploring Peer Review Dynamics with LLM Agents. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*. https://aclanthology.org/2024.emnlp-main.120/

[30] Jiazheng Kang, Mingming Ji, Zhe Zhao, and Ting Bai. 2025. Memory OS of AI Agent. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 25961–25970.

[31] LangChain Inc. 2026. LangGraph: A Library for Building Multi-Agent Workflows with LLMs. https://docs.langchain.com/oss/python/langgraph/. Accessed: January 2026.

[32] Langflow.ai. 2026. Langflow: A Powerful Tool for Building and Deploying AI-powered Agents and Workflows. https://github.com/langflow-ai/langflow. Accessed: January 2026.

[33] Stefan Larson, Anish Mahendran, Joseph J. Peper, Christopher Clarke, et al. 2019. An Evaluation Dataset for Intent Classification and Out-of-Scope Prediction. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing and the International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. 1311–1316. https://aclanthology.org/D19-1131/

[34] Der-Tsai Lee and Arthur K Lin. 1986. Generalized Delaunay triangulation for planar graphs. *Discrete & Computational Geometry* 1, 3 (1986), 201–217.

[35] Lun Li, David L. Alderson, John Doyle, and Walter Willinger. 2005. Towards a Theory of Scale-Free Graphs: Definition, Properties, and Implications. *Internet Mathematics* 2, 4 (2005), 431–523. https://doi.org/10.1080/15427951.2005.10129111

[36] Raymond Li, Samira Ebrahimi Kahou, Hannes Schulz, Vincent Michalski, Laurent Charlin, and Chris Pal. 2018. Towards Deep Conversational Recommendations. In *Proceedings of the Thirty-First Conference on Neural Information Processing Systems (NeurIPS)*, Vol. 31.

[37] Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, and et al. 2023. Holistic Evaluation of Language Models. *Transactions on Machine Learning Research (TMLR)* (2023). https://openreview.net/pdf?id=iO4LZibEqW

[38] Xingkun Liu, Arash Eshghi, Pawel Swietojanski, and Verena Rieser. 2021. Benchmarking Natural Language Understanding Services for Building Conversational Agents. In *Proceedings of the Tenth International Workshop on Spoken Dialogue Systems (IWSDS): Increasing Naturalness and Flexibility in Spoken Dialogue Interaction*. 165–183. https://doi.org/10.1007/978-981-15-9323-9_15

[39] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. 2024. AgentBench: Evaluating LLMs as Agents. *Proc. International Conference on Learning Representations (ICLR)* (2024). https://openreview.net/pdf?id=zAdUB0aCTQ

[40] Zhiwei Liu, Weiran Yao, Jianguo Zhang, Le Xue, Shelby Heinecke, Rithesh RN, Yihao Feng, Zeyuan Chen, Juan Carlos Niebles, Devansh Arpit, et al. 2024. BOLAA: Benchmarking and Orchestrating LLM Autonomous Agents. *ICLR Workshop on Large Language Model (LLM) Agents* (2024). https://openreview.net/pdf?id=BUa5ekiHlQ

[41] Gregoire Mialon, Clementine Fourrier, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2024. GAIA: A Benchmark for General AI Assistants. In *Proceedings of the Twelfth International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=fibxvahvs3

[42] n8n.io. 2026. n8n: Fair-code workflow automation platform with native AI capabilities. https://github.com/n8n-io/n8n. Accessed: January 2026.

[43] Dang Nguyen, Viet Dac Lai, Seunghyun Yoon, Ryan A. Rossi, et al. 2024. DynaSaur: Large Language Agents Beyond Predefined Actions. *Computing Research Repository (CoRR)* abs/2411.01747 (2024). https://www.paperdigest.org/?paper_

id=arxiv-2411.01747

[44] Reham Omar, Abdelghny Orogat, Ibrahim Abdelaziz, Omij Mangukiya, Panos Kalnis, and Essam Mansour. 2026. Chatty-KG: A Multi-Agent AI System for On-Demand Conversational Question Answering over Knowledge Graphs. *Proc. ACM Manag. Data (SIGMOD)* (2026).

[45] Yasmine Omri, Ziyu Gan, Zachary Broveak, Robin Geens, and et al. 2026. Agent Memory: Characterization and System Implications of Stateful Long-Horizon Workloads. *arXiv preprint arXiv:2606.06448* (2026). https://arxiv.org/abs/2606.06448

[46] OpenAI. 2025. GPT-OSS-120B and GPT-OSS-20B Model Card. https://arxiv.org/abs/2508.10925

[47] OpenAI. 2026. OpenAI Agents SDK: A Python framework for building and orchestrating multi-agent systems. https://openai.github.io/openai-agents-python/. Accessed: January 2026.

[48] OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, et al. 2024. GPT-4 Technical Report. *Computing Research Repository (CoRR)* abs/2303.08774 (2024). https://arxiv.org/abs/2303.08774

[49] Maximilian Puelma Touzel, Sneheel Sarangi, Austin Welch, Gayatri K., et al. 2024. Simulation System Towards Solving Societal-Scale Manipulation. In *Proceedings of the Workshop on Socially Responsible Language Modelling Research*. https://openreview.net/forum?id=fVl2Dhn4Kr

[50] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, et al. 2024. ChatDev: Communicative Agents for Software Development. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*. 15174–15186. https://aclanthology.org/2024.acl-long.810/

[51] Yujia Qin, Shengding Hu, Yankai Lin, Weize Chen, et al. 2024. Tool Learning with Foundation Models. *Comput. Surveys* 57, 4 (2024). https://doi.org/10.1145/3704435

[52] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, et al. 2024. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-World APIs. In *Proceedings of the Twelfth International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=dHng2O0Jjr

[53] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. 2025. Zep: A Temporal Knowledge Graph Architecture For Agent Memory. *arXiv Preprint* (2025). https://arxiv.org/pdf/2501.13956

[54] Zhengliang Shi, Yuhan Wang, Lingyong Yan, Pengjie Ren, Shuaiqiang Wang, Dawei Yin, and Zhaochun Ren. 2025. Retrieval Models Aren't Tool-Savvy: Benchmarking Tool Retrieval for Large Language Models. *arXiv preprint arXiv:2503.01763* (2025). https://arxiv.org/abs/2503.01763

[55] Significant Gravitas. 2026. AutoGPT: An open-source autonomous AI agent framework. https://github.com/Significant-Gravitas/AutoGPT. Accessed: January 2026.

[56] Alon Talmor, Jonathan Herzig, Nicholas Lourie, and Jonathan Berant. 2019. CommonsenseQA: A Question Answering Challenge Targeting Commonsense Knowledge. In *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*. 4149–4158. https://aclanthology.org/N19-1421/

[57] Wei Tao, Yucheng Zhou, Yanlin Wang, Wenqiang Zhang, Hongyu Zhang, and Yu Cheng. 2024. MAGIS: LLM-Based Multi-Agent Framework for GitHub Issue Resolution. In *Proceedings of the Thirty-Eighth Conference on Neural Information Processing Systems (NeurIPS)*. https://openreview.net/forum?id=qevq3FZ63J

[58] Brian Uzzi, Luis A. N. Amaral, and Felix Reed-Tsochas. 2007. Small-World Networks and Management Science Research: A Review. *European Management Review* 4, 2 (2007), 77–91. https://onlinelibrary.wiley.com/doi/epdf/10.1057/palgrave.emr.1500078

[59] Alexander Sasha Vezhnevets, John P. Agapiou, Avia Aharon, Ron Ziv, et al. 2023. Generative Agent-Based Modeling with Actions Grounded in Physical, Social, or Digital Space Using Concordia. *arXiv preprint arXiv:2312.03664* (2023). https://doi.org/10.48550/arXiv.2312.03664

[60] Yu Wang and Xi Chen. 2025. Mirix: Multi-Agent Memory System For LLM-Based Agents. *arXiv Preprint* (2025). https://arxiv.org/pdf/2507.07957

[61] Di Wu, Hongyu Wang, Wenhao Yu, Yuchen Zhang, Kai-Wei Chang, and Dong Yu. 2025. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. In *Proceedings of the International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=pZiyCaVuti

[62] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, et al. 2024. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversations. In *Proceedings of the First Conference on Language Modeling (COLM)*. https://openreview.net/forum?id=BAakY1hNKS

[63] Tianbao Xie, Fan Zhou, Zhoujun Cheng, Peng Shi, et al. 2024. OpenAgents: An Open Platform for Language Agents in the Wild. In *Proceedings of the ICLR Workshop on Large Language Model Agents*. https://openreview.net/forum?id=m2WwROxCcB

[64] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-Mem: Agentic Memory for LLM Agents. In *Proceedings of the Thirty-Ninth Conference on Neural Information Processing Systems (NeurIPS)*. https://openreview.net/forum?id=FiM0M8gcct

[65] Zhe Xu, Jiasheng Ye, Xiaoran Liu, Xiangyang Liu, et al. 2025. DetectiveQA: Evaluating Long-Context Reasoning on Detective Novels. In *Proceedings of the Workshop on Reasoning and Planning for Large Language Models*. https://openreview.net/forum?id=9ExIs5ELlk

[66] Ceyao Zhang, Kaijie Yang, Siyi Hu, Zihao Wang, et al. 2024. ProAgent: Building Proactive Cooperative Agents with Large Language Models. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*. 17591–17599. https://ojs.aaai.org/index.php/AAAI/article/view/29710

[67] Xinrong Zhang, Yingfa Chen, Shengding Hu, Zihang Xu, Junhao Chen, Moo Hao, Xu Han, Zhen Thai, Shuo Wang, Zhiyuan Liu, et al. 2024. ∞ Bench: Extending long context evaluation beyond 100k tokens. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 15262–15277.

[68] Zexuan Zhong, Zhengxuan Wu, Christopher D. Manning, Christopher Potts, and Danqi Chen. 2023. MQuAKE: Assessing Knowledge Editing in Language Models via Multi-Hop Questions. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*. https://openreview.net/forum?id=0hTPJBnncc

[69] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. 2024. WebArena: A Realistic Web Environment for Building Autonomous Agents. *Proc. International Conference on Learning Representations (ICLR)* (2024). https://openreview.net/pdf?id=oKn9c6ytLx